

Welcome to SEAS Online at George Washington University

Class will begin shortly

Audio: To eliminate background noise, please be sure your audio is muted. To speak, please click the hand icon at the bottom of your screen (**Raise Hand**). When instructor calls on you, click microphone icon to unmute. When you've finished speaking, ***be sure to mute yourself again.***

Chat: Please type your questions in Chat.

Recordings: As part of the educational support for students, we provide downloadable recordings of each class session to be used exclusively by registered students in that particular class. **Releasing these recordings is strictly prohibited.**

SEAS 8515 – Lecture 3

Data Eng. Lifecycle & Spark

Walid Hassan, M.B.A, D.Eng.

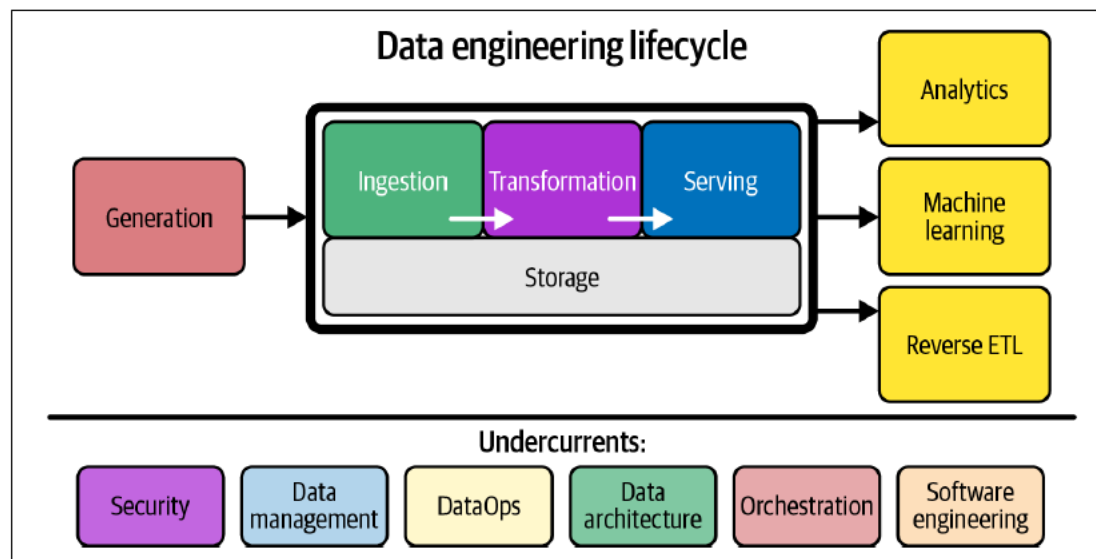
Data Engineering Defined

- Data engineering is the development, implementation, and maintenance of systems and processes that take in raw data and produce high-quality, consistent information that supports downstream use cases, such as analysis and machine learning.
- Data engineering is the intersection of security, data management, DataOps, data architecture, orchestration, and software engineering.
- A data engineer manages the data engineering lifecycle, beginning with getting data from source systems and ending with serving data.

Src: Fundamentals of Data Engineering (Reis & Housley)

© Walid Hassan, M.B.A, D.Eng.

Data Engineering Defined

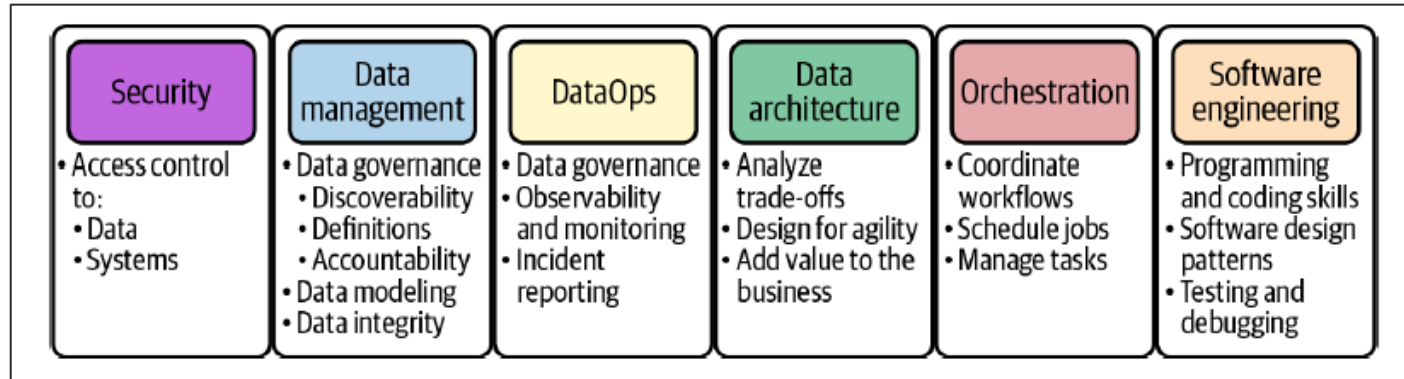


- Generation
- Storage
- Ingestion
- Transformation
- Serving

Src: Fundamentals of Data Engineering (Reis & Housley)

© Walid Hassan, M.B.A, D.Eng.

Data Engineering Defined



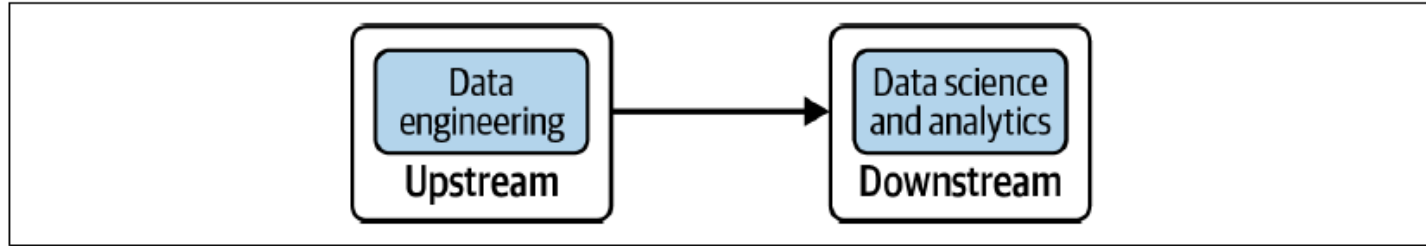
The data engineering lifecycle also has a notion of undercurrents – critical ideas across the entire lifecycle.

These include security, data management, DataOps, data architecture, orchestration, and software engineering

Src: Fundamentals of Data Engineering (Reis & Housley)

© Walid Hassan, M.B.A, D.Eng.

Data Engineer vs. Scientist

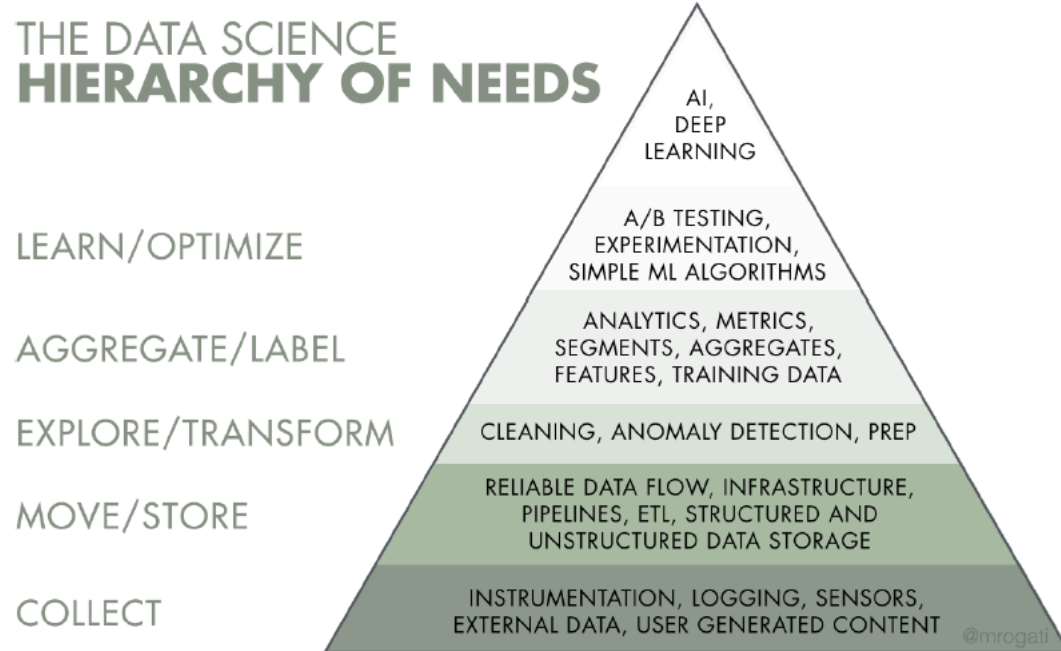


- Data engineering is separate from data science and analytics.
- Data engineering sits upstream from data science, meaning data engineers provide the inputs used by data scientists (downstream) who convert these inputs into something useful.

Src: Fundamentals of Data Engineering (Reis & Housley)

© Walid Hassan, M.B.A, D.Eng.

Data Science Hierarchy of Needs

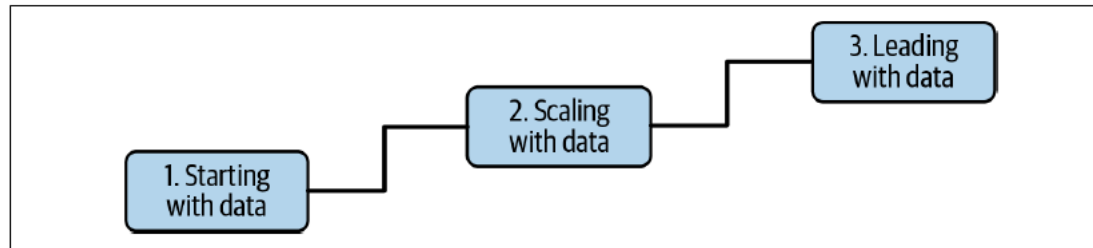


An estimated 70% to 80% of their (data scientist) time is spent toiling in the bottom three parts of the hierarchy.

Src: Fundamentals of Data Engineering (Reis & Housley)

© Walid Hassan, M.B.A, D.Eng.

Data Maturity



- Data maturity is the progression toward higher data utilization, capabilities, and integration across the organization
- It does not simply depend on the age or revenue of a company.

Src: Fundamentals of Data Engineering (Reis & Housley)

© Walid Hassan, M.B.A, D.Eng.

Data Maturity

Stage 1:

- The company may have fuzzy, loosely defined goals or no goals.
- Data architecture and infrastructure are in the very early stages of planning and Development.
- The practicalities of getting value from data are typically poorly understood.
- Reports or analyses lack formal structure, and most requests for data are ad hoc.

Data Maturity

Stage 2:

- Company has moved away from ad hoc data requests and has formal data practices.
- Now the challenge is creating scalable data architectures and planning for a future where the company is genuinely data-driven.
- Data engineering roles move from generalists to specialists, with people focusing on particular aspects of the data engineering lifecycle.

Src: Fundamentals of Data Engineering (Reis & Housley)

© Walid Hassan, M.B.A, D.Eng.

Data Maturity

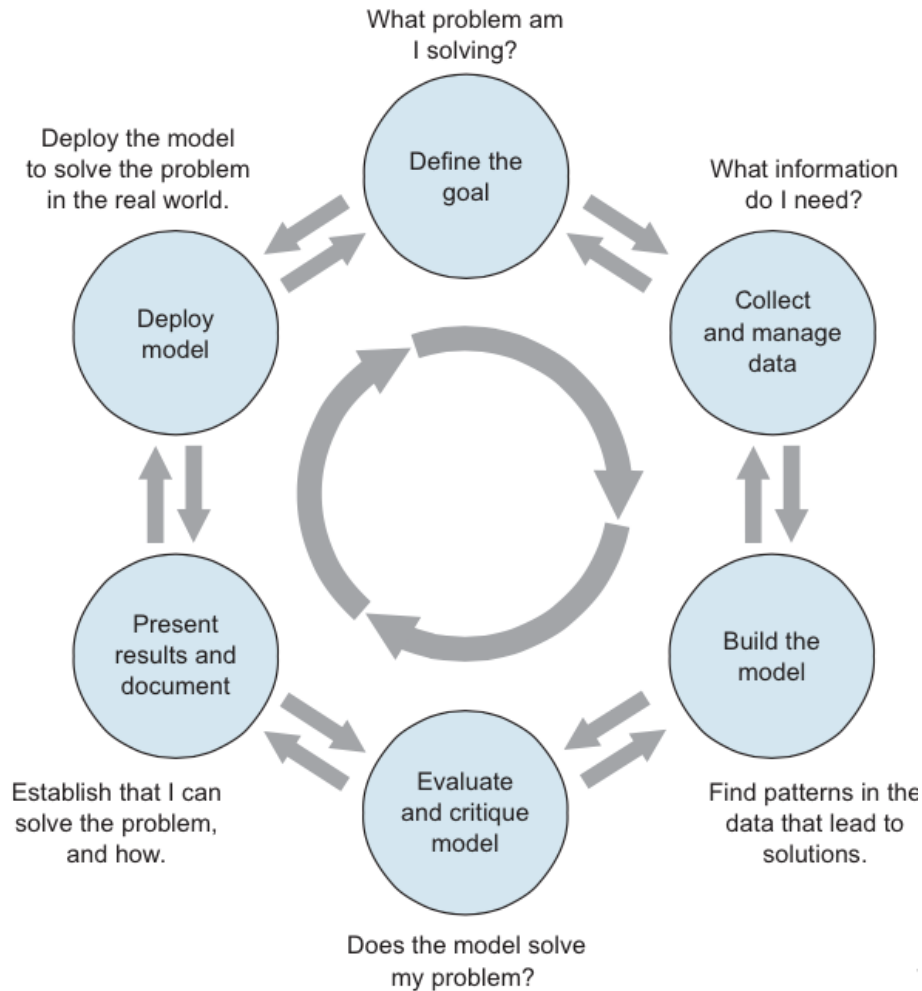
Stage 3:

- The company is data-driven.
- The automated pipelines and systems created by data engineers allow people within the company to do self-service analytics and ML.
- Introducing new data sources is seamless, and tangible value is derived.
- Data engineers implement proper controls and practices to ensure that data is always available to the people and systems & their roles continue to specialize more deeply than in stage 2.

Src: Fundamentals of Data Engineering (Reis & Housley)

© Walid Hassan, M.B.A, D.Eng.

Data Science Workflow



Src: <https://www.hcbravo.org/IntroDataSci/bookdown-notes/introduction-and-overview.html>

© Walid Hassan, M.B.A, D.Eng.

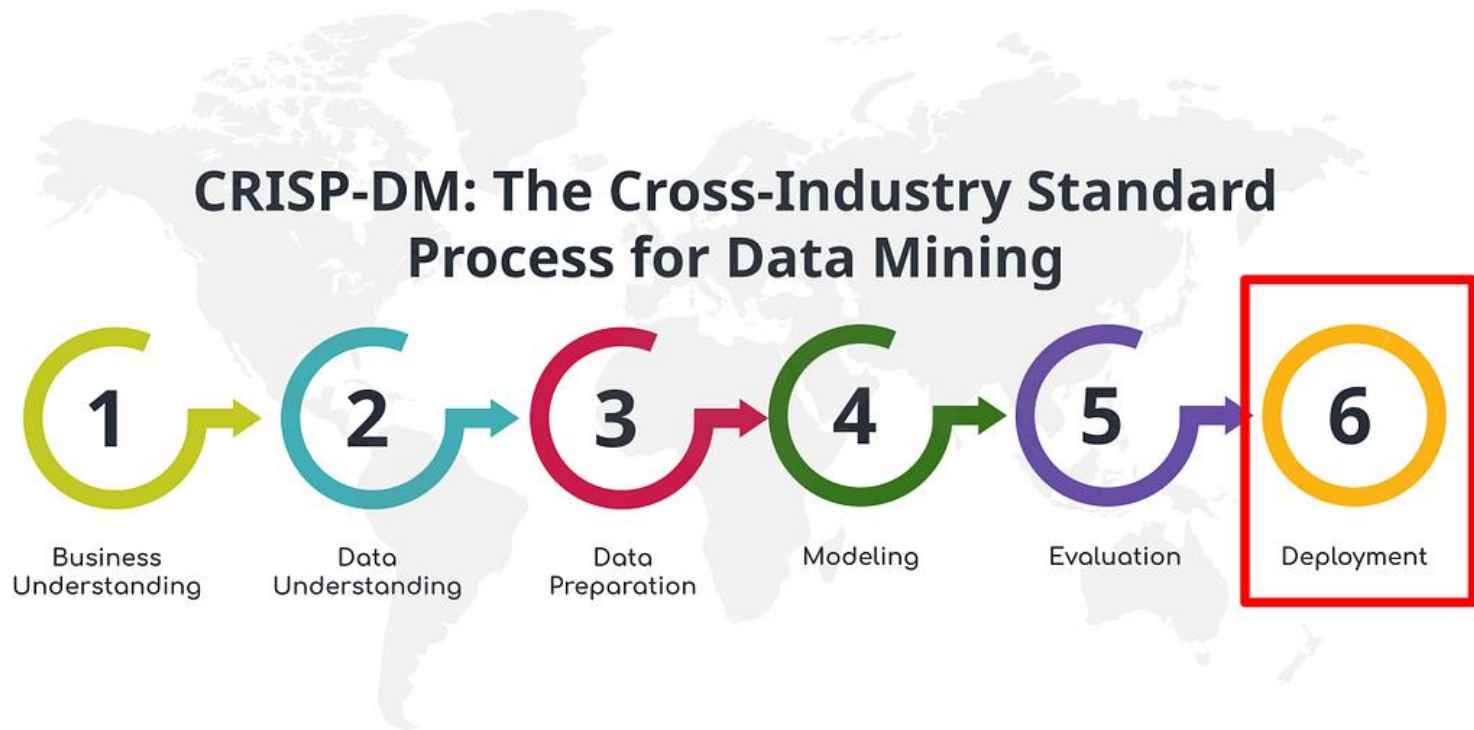
OSEMN (awesome)



Src: <https://www.datascience-pm.com/osemn/>

© Walid Hassan, M.B.A, D.Eng.

CRISP-DM

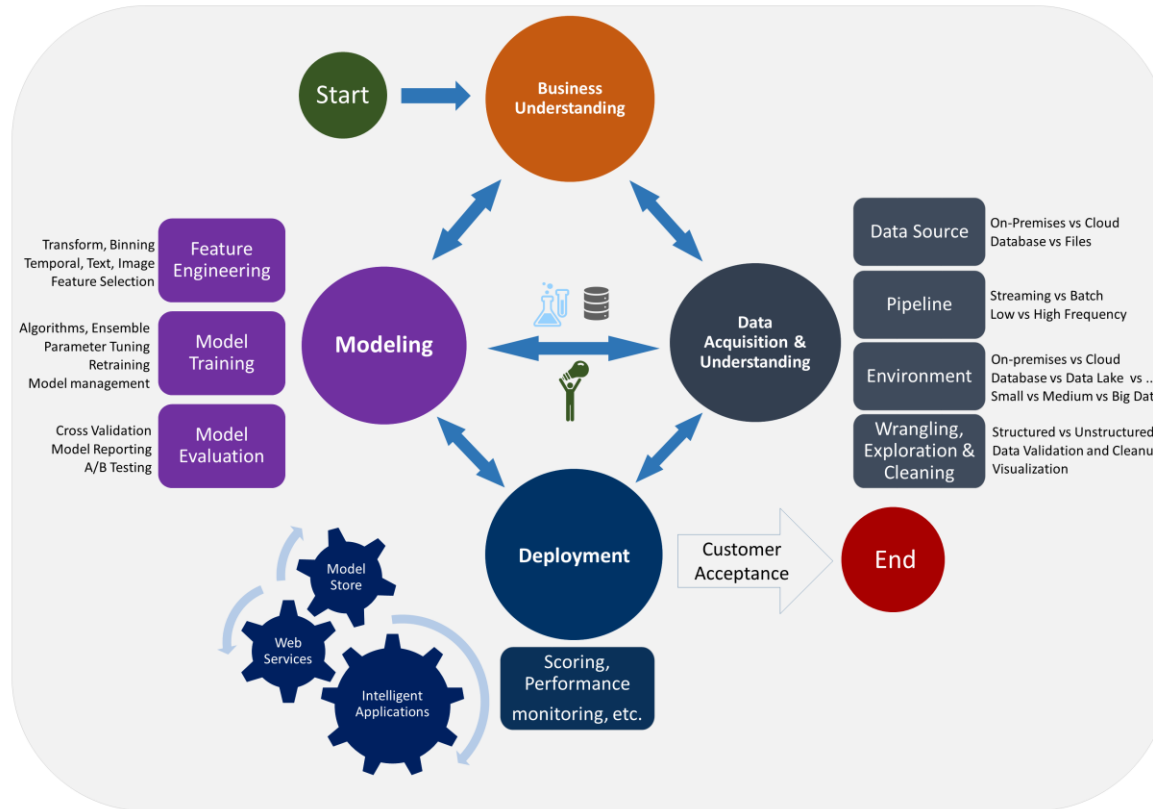


Src:<https://medium.com/@yennhi95zz/6-the-deployment-phase-of-the-crisp-dm-methodology-a-detailed-discussion-f802a7cb9a0f>

© Walid Hassan, M.B.A, D.Eng.

TDSP (Team Data Science Process)

Data Science Lifecycle



Src: Microsoft

© Walid Hassan, M.B.A, D.Eng.

TDSP (Teams Data Science Process)

- Data science lifecycle definition
- Standardized project structure
- Infrastructure and resources that are ideal for data science projects
- **Responsible AI:** and a commitment to the advancement of AI, driven by ethical principles

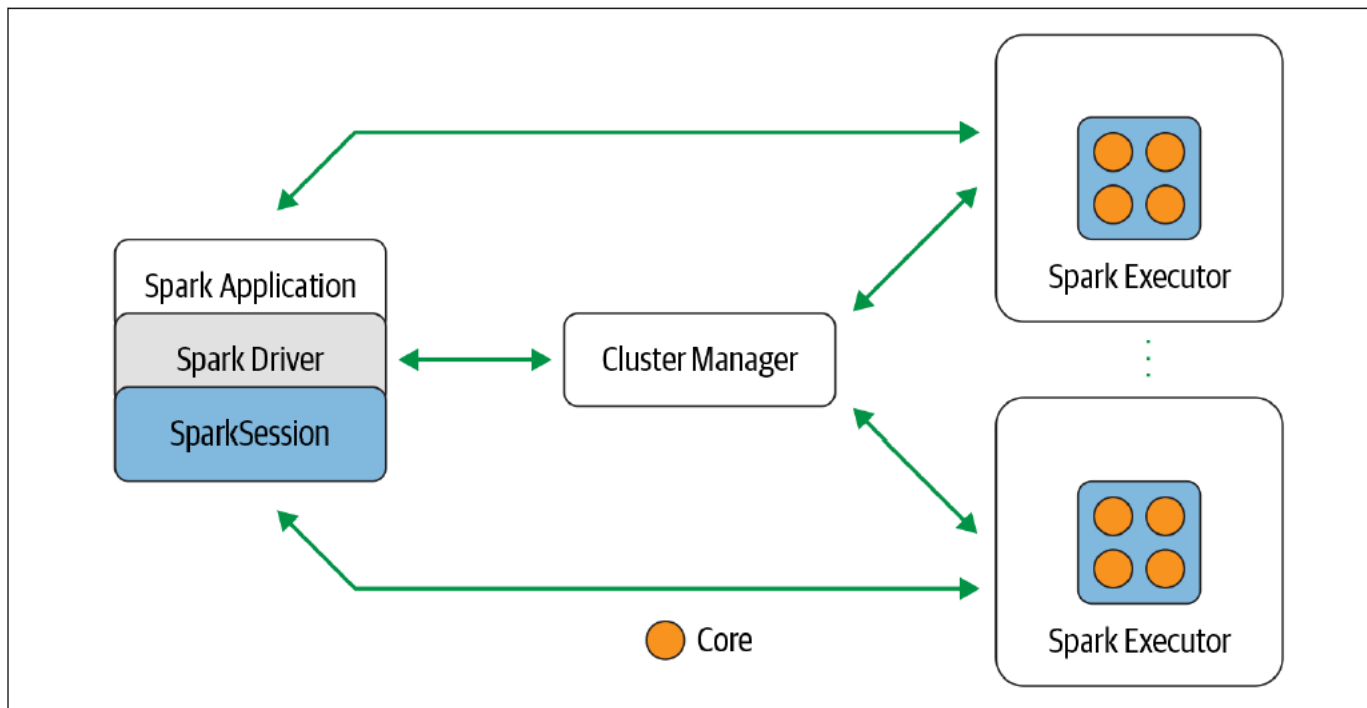
Src: Microsoft; <https://learn.microsoft.com/en-us/azure/architecture/data-science-process/overview>

© Walid Hassan, M.B.A, D.Eng.

Data Science Lifecycle & Spark

Stage	Spark's Role
Data Collection	Reading structured and unstructured data from HDFS, S3, JDBC, Kafka, etc.
Data Preparation	RDD/DataFrame APIs for cleaning, transforming, and aggregating large datasets.
Modeling	Distributed machine learning with MLlib, integration with external AI libraries.
Evaluation	Large-scale evaluation with Spark SQL, distributed metrics computation.
Deployment	Model serving with Spark Structured Streaming or integration with pipelines.

Key Definitions/Concepts in Spark



Src: Learning Spark

© Walid Hassan, M.B.A, D.Eng.

Key Definitions/Concepts in Spark

Application: A user program built on Spark using its APIs. It consists of a driver program and executors on the cluster.

SparkSession:

- An object that provides a point of entry to interact with underlying Spark functionality and allows programming Spark with its APIs.
- In an interactive Spark shell, the Spark driver instantiates a SparkSession for you, while in a Spark application, you create a SparkSession object yourself.

```
from pyspark.sql import SparkSession  
  
# Initialize Spark session  
spark =  
SparkSession.builder.appName("MyApp").getOrCreate()
```

Src: Learning Spark

© Walid Hassan, M.B.A, D.Eng.

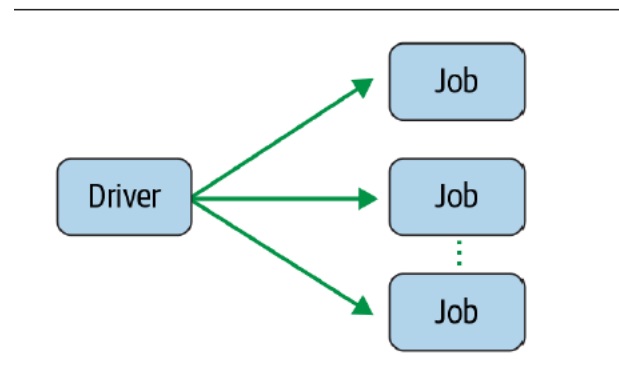
Key Definitions/Concepts in Spark

- **Job**
 - A parallel computation consisting of multiple tasks that gets spawned in response to a Spark action (e.g., `save()`, `collect()`).
- **Stage:**
 - Each job gets divided into smaller sets of tasks called stages that depend on each other.
- **Task**
 - A single unit of work or execution that will be sent to a Spark executor.

Key Definitions/Concepts in Spark

Driver:

- Instantiates a `SparkSession`,
- It communicates with the cluster manager; it requests resources (CPU, memory, etc.) from the cluster manager for Spark's executors (JVMs);
- Transforms all the Spark operations into DAG computations, schedules them, and distributes their execution as tasks across the Spark executors.
- Once the resources are allocated, it communicates directly with the executors.



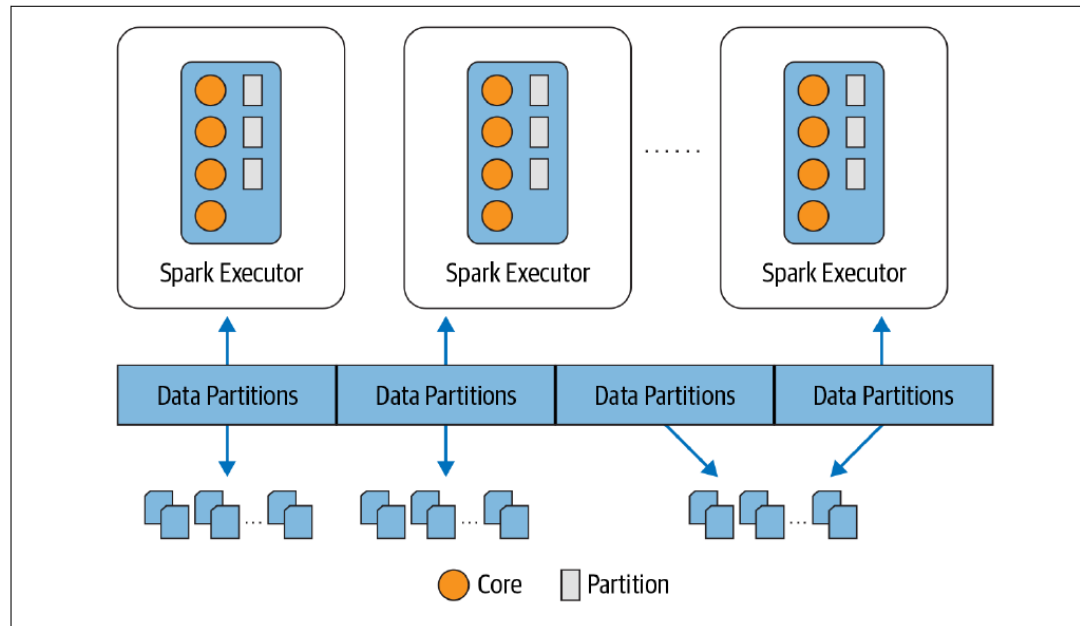
Key Definitions/Concepts in Spark

- **SparkSession:**
- It is a unified conduit to all Spark operations and data.
- Through it, you can create JVM runtime parameters, define Data-Frames and Datasets, read from data sources, access catalog metadata, and issue Spark SQL queries.
- SparkSession is created using one of the high-level APIs in the programming language of your choice.

Key Definitions/Concepts in Spark

- **Cluster Manager (Resource Manager):**
 - The cluster manager is responsible for managing and allocating resources for the cluster of nodes on which your Spark application runs. Currently, Spark supports four cluster managers: the built-in standalone cluster manager, Apache Hadoop YARN, Apache Mesos, and Kubernetes.
- **Spark Executor:**
 - A Spark executor runs on each worker node in the cluster. The executors communicate with the driver program and are responsible for executing tasks on the workers. In most deployments modes, only a single executor runs per node.

Key Definitions/Concepts in Spark



This code snippet will break up the physical data stored across clusters into eight partitions, and each executor will get one or more partitions to read into its memory:

```
log_df = spark.read.text("path_to_large_text_file").repartition(8)
print(log_df.rdd.getNumPartitions())
```

Src: Learning Spark

© Walid Hassan, M.B.A, D.Eng.

Directed Acyclic Graph

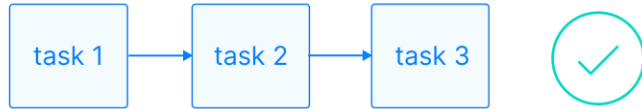
- Directed: There is a clear direction of flow between tasks.
- Acyclic: No circular dependencies are allowed; a task cannot depend on itself or on a task that ultimately depends on it. This acyclic property is extremely important, as it prevents us from running into circular dependencies.
- Graph: The structure allows for easy visualization and conceptualization of the workflow. As such, we need to make sure that this implicit task order is also enforced when running this data process.

Src: <https://www.astronomer.io/docs/learn/dags/>

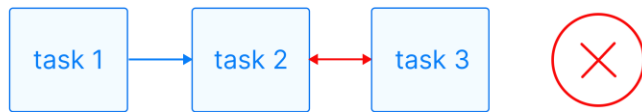
© Walid Hassan, M.B.A, D.Eng.

Directed Acyclic Graph

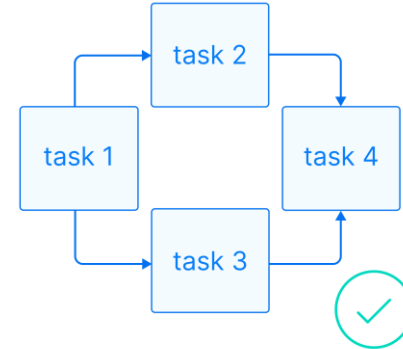
Directed



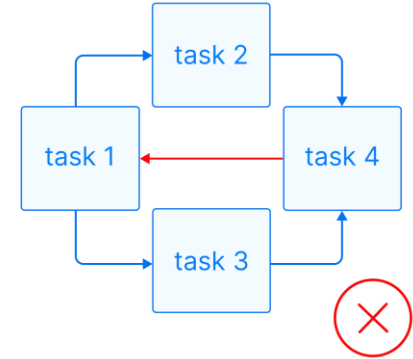
Not directed



Acyclic



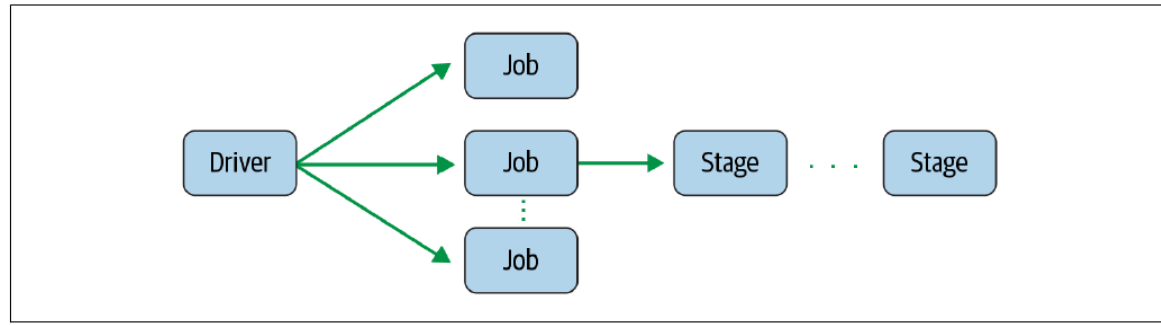
Not Acyclic



Src: <https://www.astronomer.io/docs/learn/dags/>

© Walid Hassan, M.B.A, D.Eng.

Key Definitions/Concepts in Spark

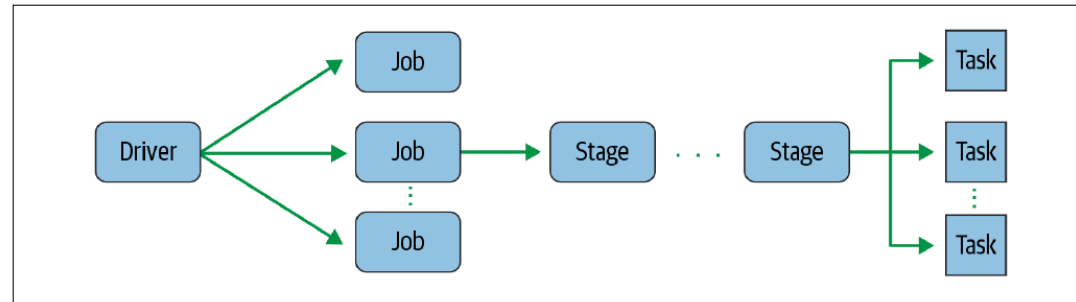


Stages:

- As part of the DAG nodes, stages are created based on what operations can be performed serially or in parallel .
- Not all Spark operations can happen in a single stage, so they may be divided into multiple stages.
- Often stages are delineated on the operator's computation boundaries, where they dictate data transfer among Spark executors.

Src: Learning Spark

Key Definitions/Concepts in Spark



Tasks :

- Each stage is comprised of Spark tasks (a unit of execution), which are then federated across each Spark executor;
- Each task maps to a single core and works on a single partition of data . As such, an executor with 16 cores can have 16 or more tasks working on 16 or more partitions in parallel, making the execution of Spark's tasks exceedingly parallel.

Transformations, Actions, and Lazy Evaluation

Spark operations on distributed data can be classified into two types:

Transformations and Actions.

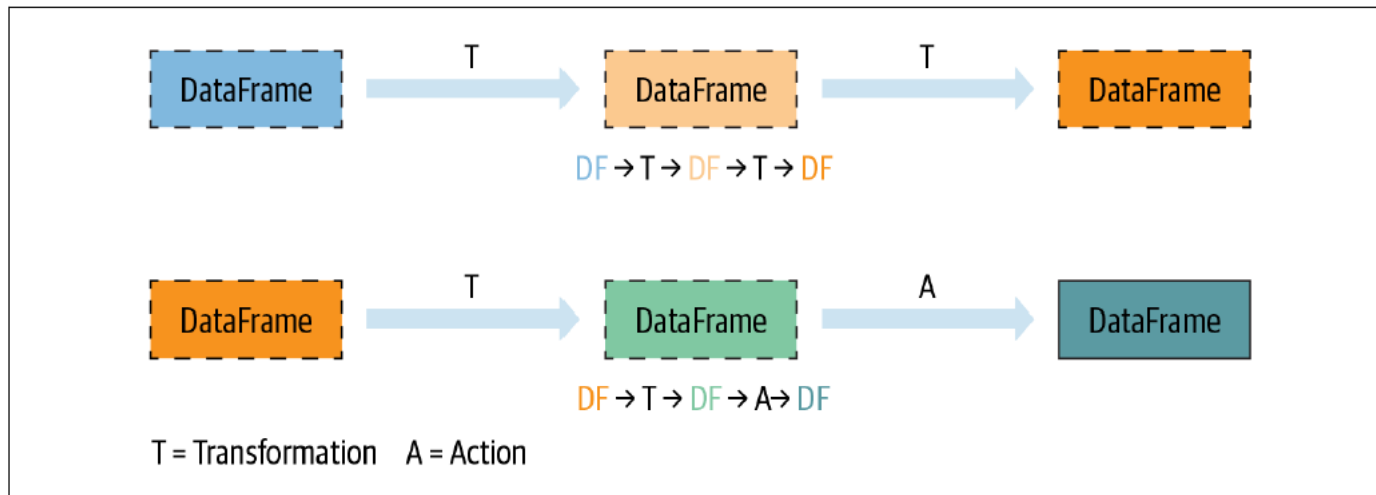
Transformations, as the name suggests, transform a Spark DataFrame into a new DataFrame without altering the original data, giving it the property of immutability. Put another way, an operation such as `select()` or `filter()` will not change the original DataFrame; instead, it will return the transformed results of the operation as a new DataFrame.

Transformations, Actions, and Lazy Evaluation

All transformations are evaluated lazily. That is, their results are not computed immediately, but they are recorded or remembered as a lineage. A recorded lineage allows Spark, at a later time in its execution plan, to rearrange certain transformations, coalesce them, or optimize transformations into stages for more efficient execution. Lazy evaluation is Spark's strategy for delaying execution until an action is invoked or data is "touched" (read from or written to disk).

Transformations, Actions, and Lazy Evaluation

An action triggers the lazy evaluation of all the recorded transformations. In Figure, all transformations T are recorded until the action A is invoked. Each transformation T produces a new DataFrame.



Src: Learning Spark

© Walid Hassan, M.B.A, D.Eng.

Transformations, Actions, and Lazy Evaluation

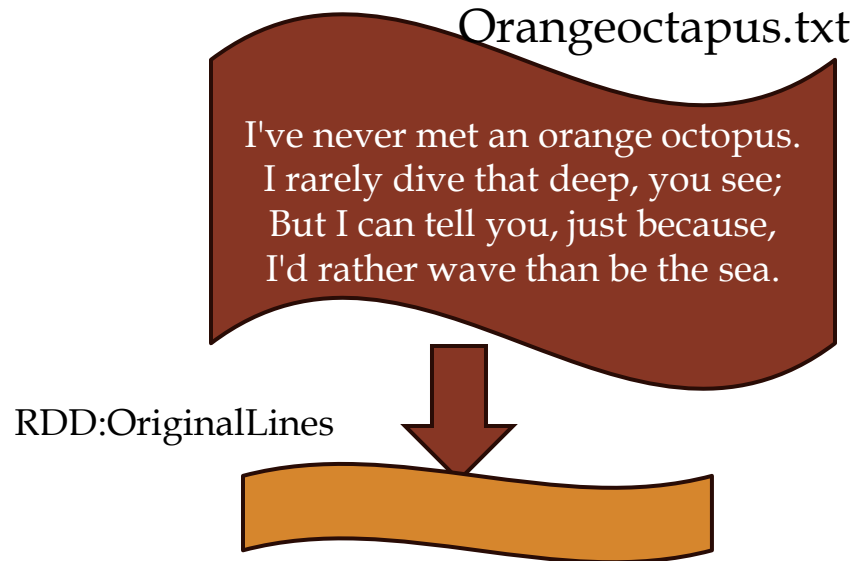
```
val originalLines = sc.textFile("orangeoctopus.txt")  
  
val uppercasedLines = originalLines.map(line =>  
  line.trim.toUpperCase)  
  
val interestingLines = uppercasedLines.filter(line =>  
  line.startsWith("I"))  
  
val resultCount = interestingLines.count()
```

Orangeoctopus.txt

I've never met an orange octopus.
I rarely dive that deep, you see;
But I can tell you, just because,
I'd rather wave than be the sea.

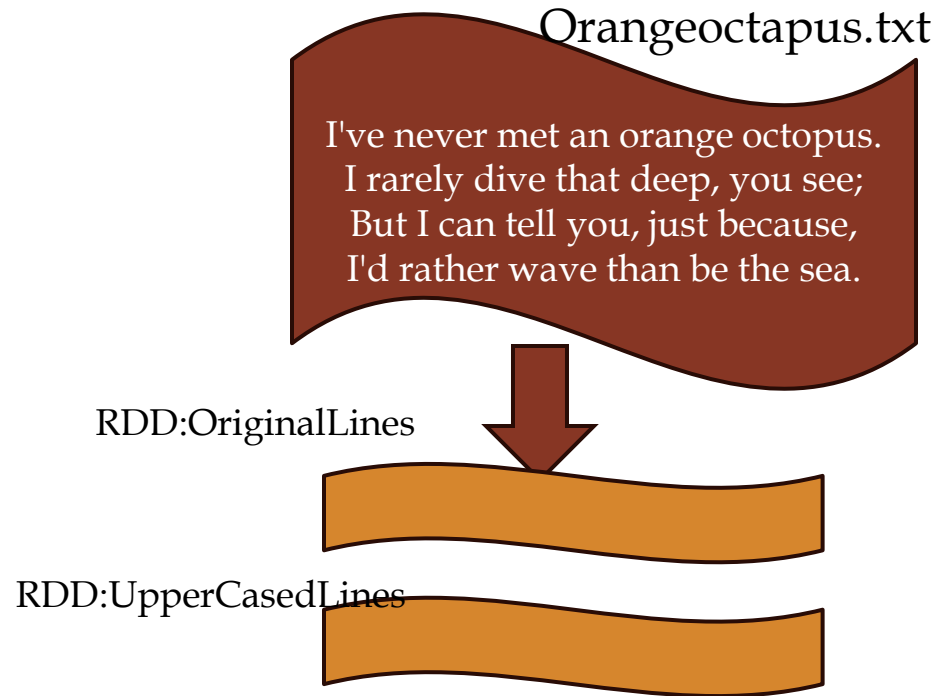
Transformations, Actions, and Lazy Evaluation

```
val originalLines = sc.textFile("orangeoctopus.txt")  
  
val uppercasedLines = originalLines.map(line =>  
  line.trim.toUpperCase)  
  
val interestingLines = uppercasedLines.filter(line =>  
  line.startsWith("I"))  
  
val resultCount = interestingLines.count()
```



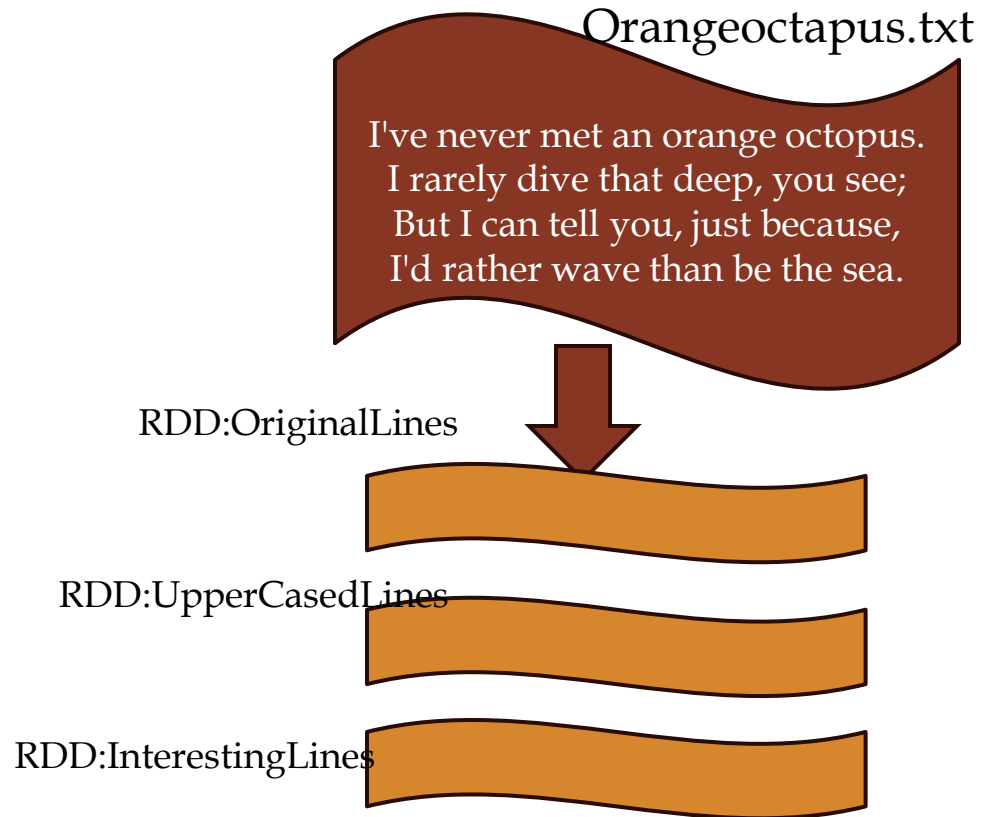
Transformations, Actions, and Lazy Evaluation

```
val originalLines = sc.textFile("orangeoctopus.txt")  
  
val uppercasedLines = originalLines.map(line =>  
  line.trim.toUpperCase)  
  
val interestingLines = uppercasedLines.filter(line =>  
  line.startsWith("I"))  
  
val resultCount = interestingLines.count()
```



Transformations, Actions, and Lazy Evaluation

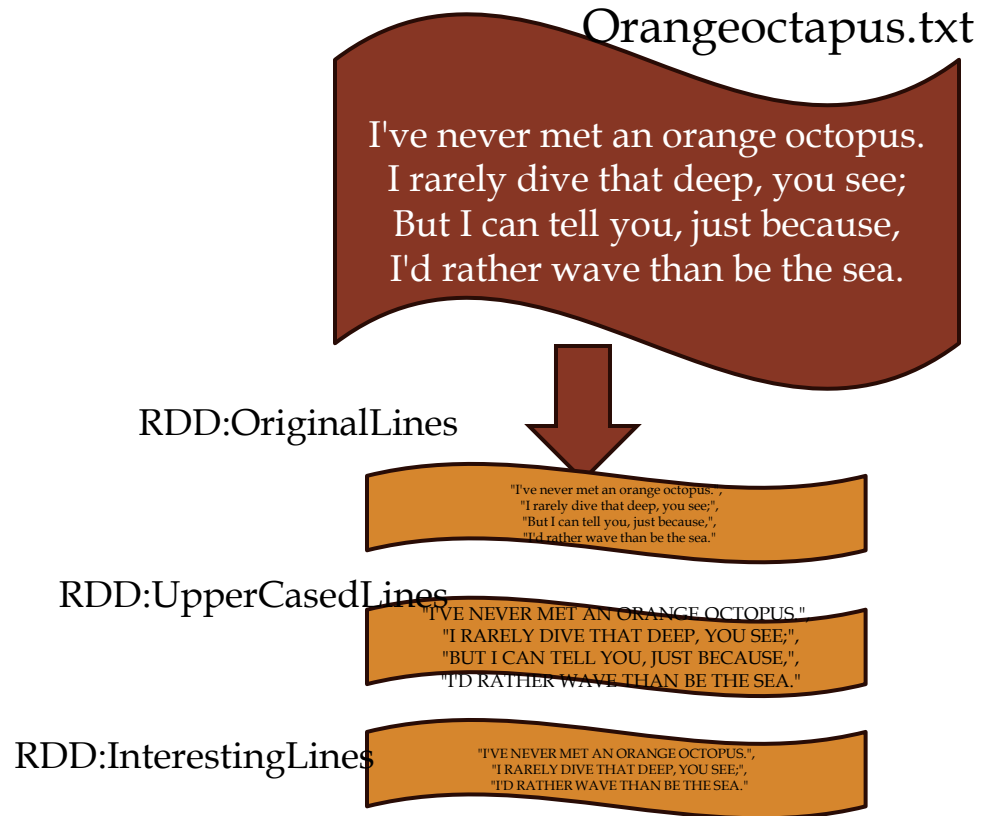
```
val originalLines = sc.textFile("orangeoctopus.txt")  
  
val uppercasedLines = originalLines.map(line =>  
  line.trim.toUpperCase)  
  
val interestingLines = uppercasedLines.filter(line =>  
  line.startsWith("I"))  
  
val resultCount = interestingLines.count()
```



Transformations, Actions, and Lazy Evaluation

```
val originalLines = sc.textFile("orangeoctopus.txt")  
  
val uppercasedLines = originalLines.map(line =>  
  line.trim.toUpperCase)  
  
val interestingLines = uppercasedLines.filter(line =>  
  line.startsWith("I"))  
  
val resultCount = interestingLines.count()
```

3



Transformations, Actions, and Lazy Evaluation

- Lazy evaluation allows Spark to optimize your queries by peeking into your chained transformations
- Lineage and data immutability provide fault tolerance.
- Because Spark records each transformation in its lineage and the DataFrames are immutable between transformations, it can reproduce its original state by simply replaying the recorded lineage, giving it resiliency in the event of failures.

Transformations	Actions
orderBy()	show()
groupBy()	take()
filter()	count()
select()	collect()
join()	save()

Transformations, Actions, and Lazy Evaluation

In Python

```
>>> strings = spark.read.text("../README.md")
```

```
>>> filtered = strings.filter(strings.value.contains("Spark"))
```

```
>>> filtered.count()
```

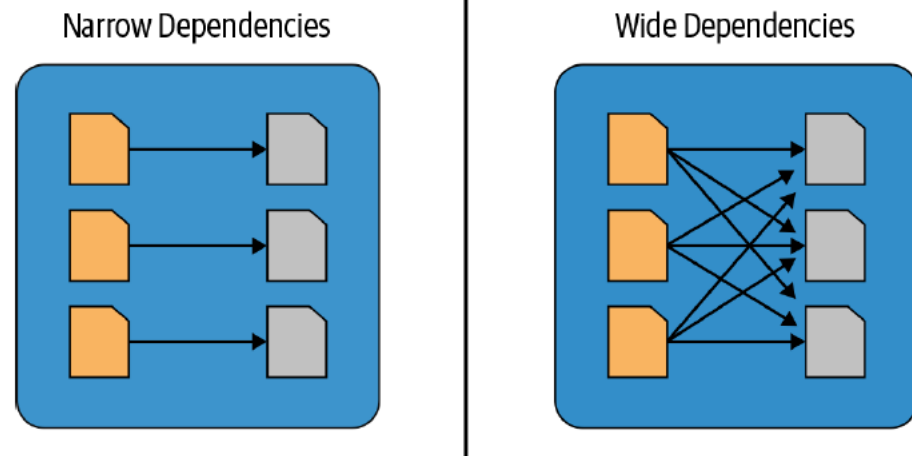
- Nothing in a query plan is executed until an **action** is invoked.
- The following example, shown both in Python and Scala, has two transformations—`read()` and `filter()`—and one action—`count()`.
- The action is what triggers the execution of all transformations recorded as part of the query execution plan.

Transformations, Actions, and Lazy Evaluation

- A huge advantage of the lazy evaluation scheme is that Spark can inspect your computational query and ascertain how it can optimize it.
- This optimization can be done by either joining or pipelining some operations and assigning them to a stage, or breaking them into stages by determining which operations require a shuffle or exchange of data across clusters.

Transformations, Actions, and Lazy Evaluation

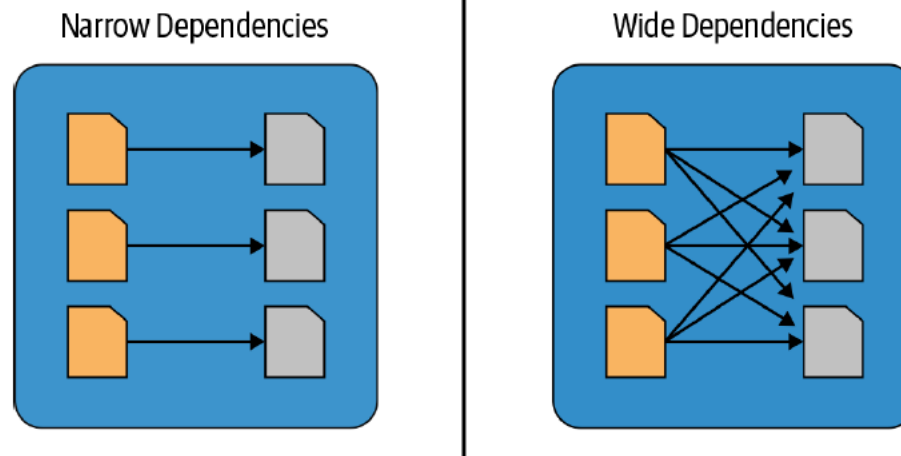
- Transformations: either narrow dependencies or wide dependencies.
- Any transformation where a single output partition can be computed from a single input partition is a narrow transformation.
- In the previous code snippet, `filter()` and `contains()` represent narrow transformations because they can operate on a single partition and produce the resulting output partition without any exchange of data.



Transformations, Actions, and Lazy Evaluation

However, `groupBy()` or `orderBy()` instruct Spark to perform wide transformations, where data from other partitions is read in, combined, and written to disk. Since each partition will have its own count of the word that contains the “Spark” word in its row of data, a `count (groupBy())` will force a shuffle of data from each of the executor’s partitions across the cluster. In this transformation, `orderBy()` requires output from other partitions to compute the final

aggregation



Src: Learning Spark

© Walid Hassan, M.B.A, D.Eng.

Transformations, Actions, and Lazy Evaluation

Job/Stages & Tasks relationship

Aspect	What Determines It?	Relation to Cores/Nodes
Jobs	Number of actions (count, show, collect, etc.).	Independent of cores/nodes.
Stages	Boundaries created by narrow/wide dependencies.	Independent of cores/nodes.
Tasks	Number of partitions of the data being processed.	Tasks run in parallel across cores/nodes.
Parallelism	Number of tasks that can run concurrently.	Determined by available cores across all nodes.

Example

```
rdd = sc.parallelize([1, 2, 3, 4, 5, 6, 7, 8], 4) # 4 partitions
```

```
result = rdd.map(lambda x: x * 2).reduce(lambda x, y: x + y)
```

- `sc.parallelize([1, 2, 3, 4, 5, 6, 7, 8], 4)` creates an RDD from the given list, splits it into 4 partitions, and distributes it across the cluster for parallel processing. (more on RDDs shortly – Resilient Distributed Datasets)

Example

Stage 1: Map Transformation (**`rdd.map(lambda x: x * 2)`**.)

Number of Tasks: 4 (one for each partition).

Each task applies the map function (`lambda x: x * 2`) independently on its partition.

Output:

Partition 1: [2, 4]

Partition 2: [6, 8]

Partition 3: [10, 12]

Partition 4: [14, 16]

Example

Stage 2: Local Reduce (**`reduce(lambda x, y: x + y)`**)

Number of Tasks: 4 (still one for each partition).

Each task performs a local reduction within its partition using the reduce function (`lambda x, y: x + y`).

This is the local aggregation step.

Partition 1: `sum([2, 4]) = 6`

Partition 2: `sum([6, 8]) = 14`

Partition 3: `sum([10, 12]) = 22`

Partition 4: `sum([14, 16]) = 30`

Example

Stage 3: Global Reduce

Number of Tasks: 1 (aggregates results across all partitions).

The single task performs a global reduction of the intermediate results:

$\text{sum}([6, 14, 22, 30]) = 72$

Summary

Stage	Description	# Tasks	Output
Stage 1: Map	Apply map to each partition	4 tasks	[2, 4], [6, 8], [10, 12], [14, 16]
Stage 2: Local Reduce	Local aggregation within partitions	4 tasks	6, 14, 22, 30
Stage 3: Global Reduce	Final aggregation across partitions	1 task	72

Spark & Partitioning

Partitions are determined by Spark itself, but the underlying storage system (e.g., HDFS, S3, or local filesystem) plays a crucial role in the partitioning process.

Spark determines how to distribute and parallelize data into partitions across the cluster, and each partition will be processed by a separate task.

The number of partitions Spark creates for a given dataset depends on:

- The data source (e.g., a file system, RDD, or DataFrame).
- How Spark is configured (e.g., default parallelism or specified partition counts).
- Spark's optimizations

Spark & Partitioning

When reading data from distributed file systems (e.g., HDFS or S3), Spark splits the data into logical partitions based on its own configuration.

For example: HDFS (Hadoop Distributed File System):

- HDFS splits large files into blocks (typically 128 MB or 256 MB by default).
- Spark can efficiently parallelize processing if its partition size is configured to align reasonably well with these block sizes

Spark & Hadoop Example

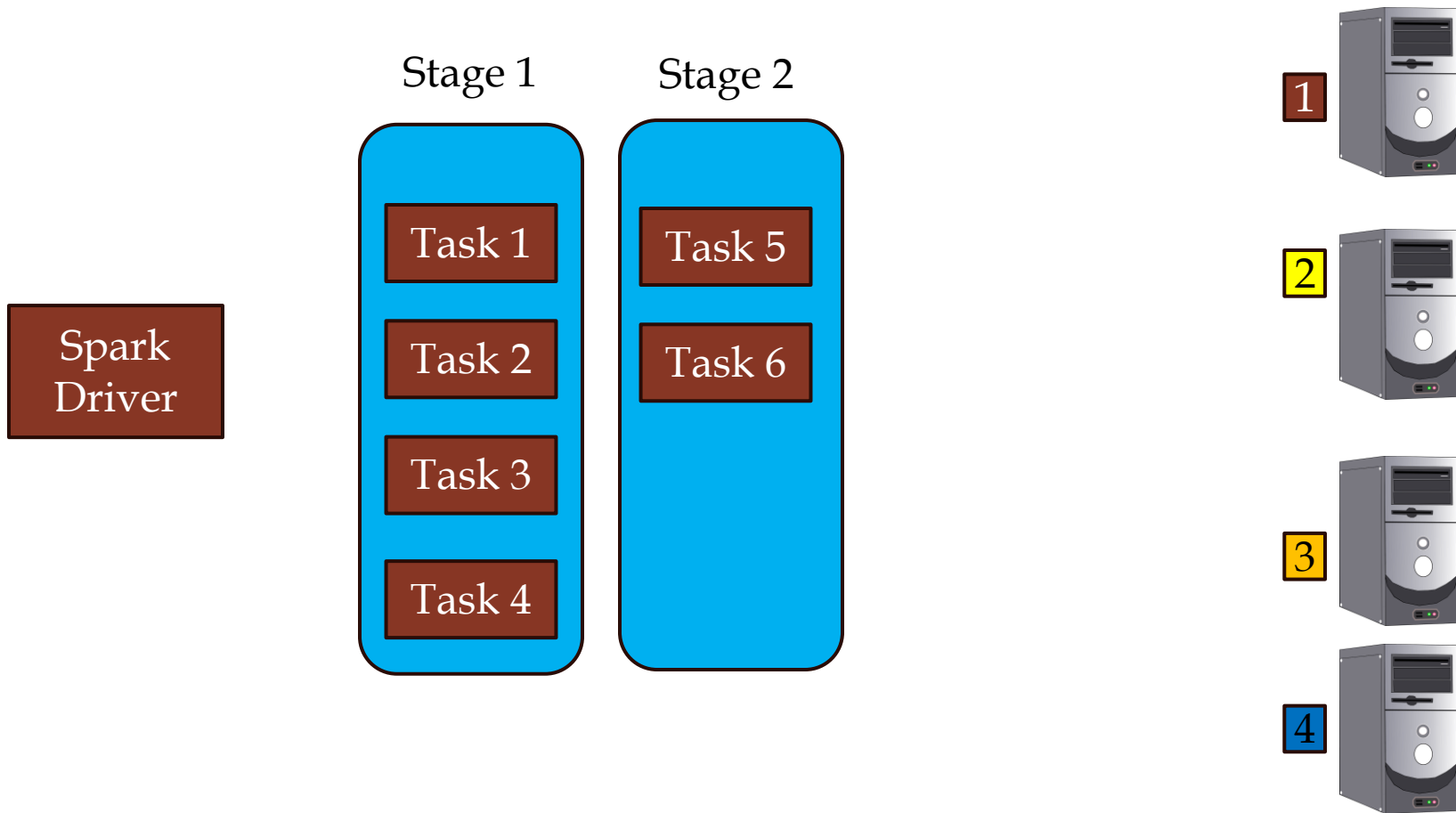
Spark wants to count the average word lengths from a file
File is in HDFS (4 blocks).

A Sample Execution of this job is as follows:

- Stage 1: On each block, execute the “counting” for that block
- Stage 2: Collect the counts and Average them out.

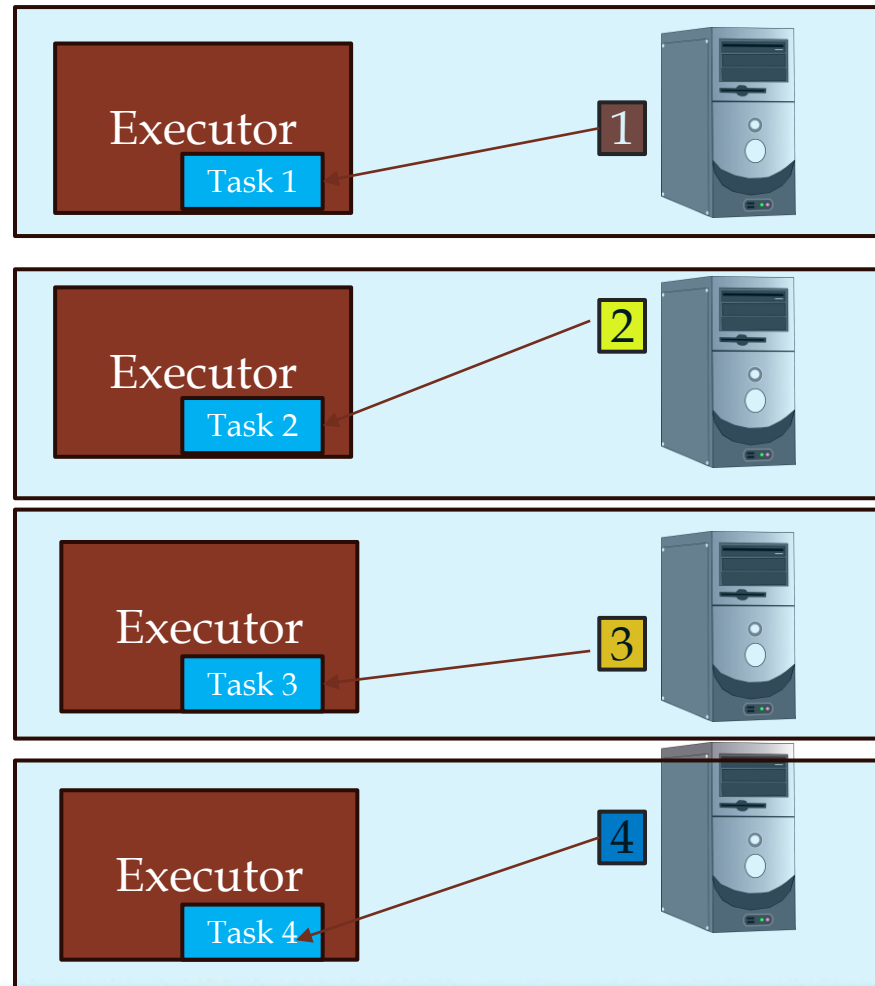


Spark & Hadoop Example

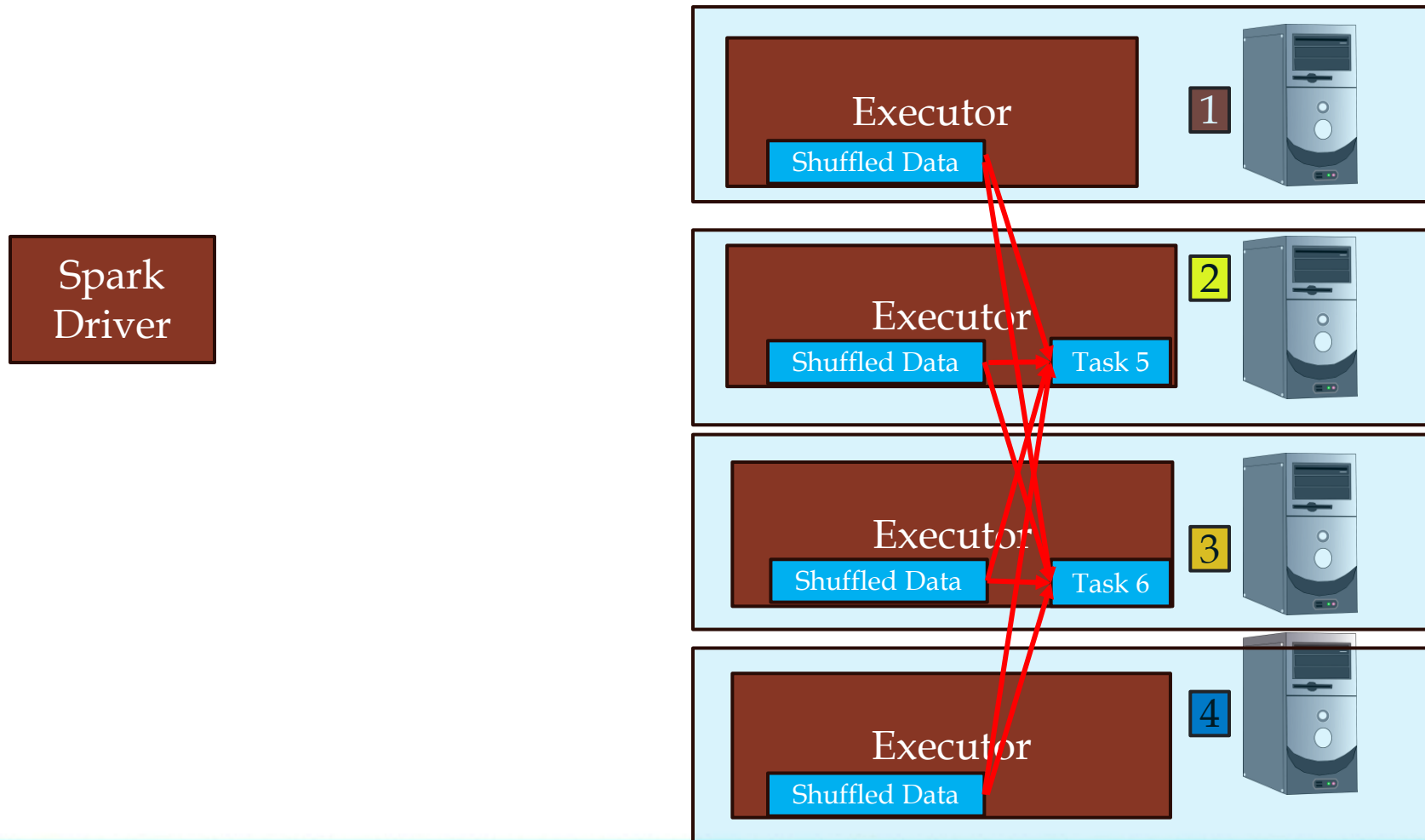


Spark & Hadoop Example

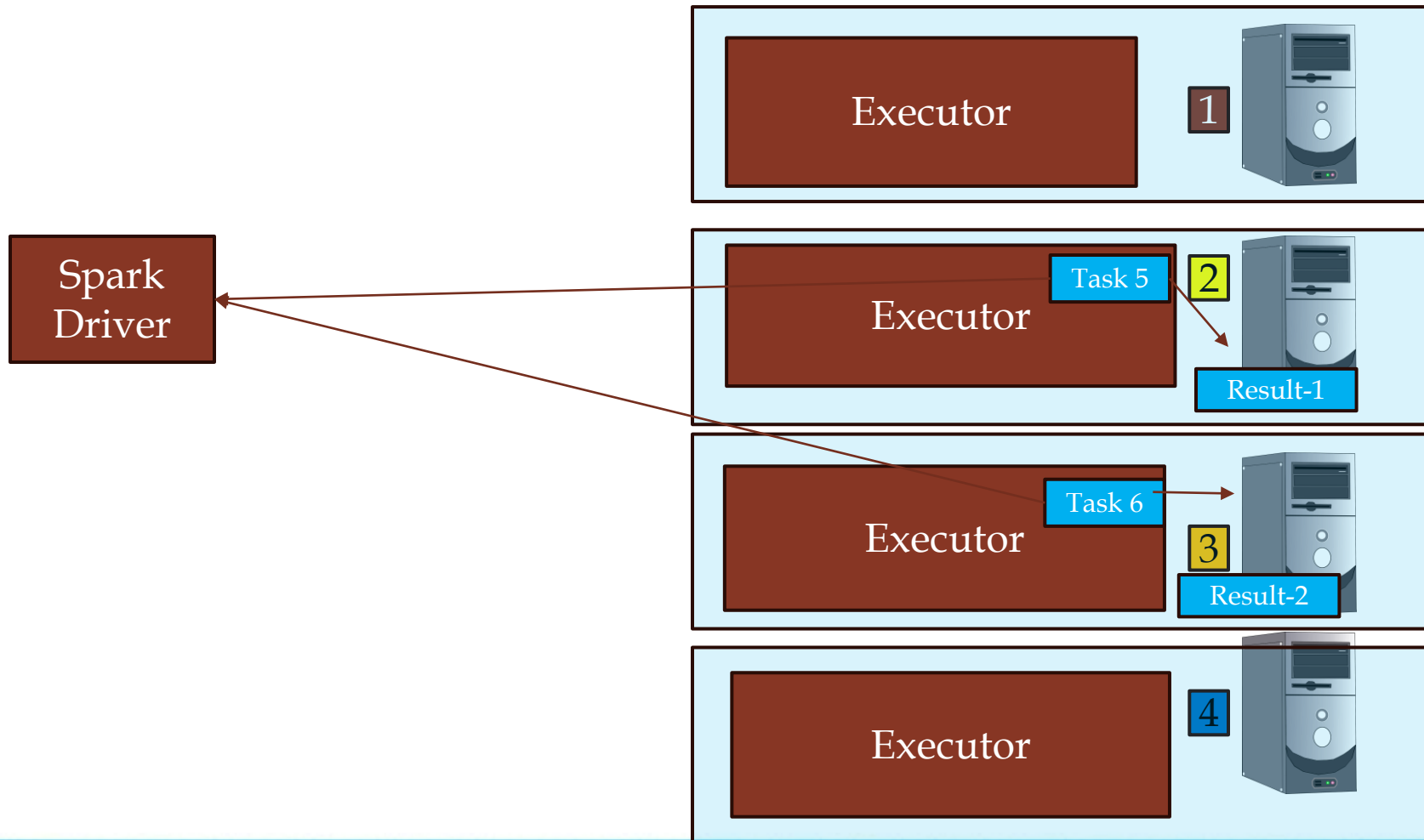
Spark
Driver



Spark & Hadoop Example



Spark & Hadoop Example



Spark & Partitioning

In Summary, Spark decides the number of partitions for data transformations and jobs, based on:

- The data source (file system, RDD, etc.).
- The size of the data and how Spark chooses to split it into smaller pieces (partitions).
- Default parallelism (based on the cluster's available resources).

Logical Partitioning

Improved Query Performance: By organizing data based on a logical dimension (e.g., state, date, customer ID), Spark jobs can process only the relevant partitions instead of scanning the entire dataset.

Efficient Data Retrieval: Partitioning allows Spark to perform partition pruning, where it skips irrelevant partitions during execution.

Parallel Processing: Partitioning ensures data is spread across multiple nodes for parallel processing.

To illustrate, Imagine you have 1 TB of sales data stored as a single file in HDFS with no logical partitioning. If you run a query to filter by state = 'CA', Spark will have to read the entire file to extract the relevant rows, leading to inefficiency.



Hands On.....
M&M



Details for Job 0

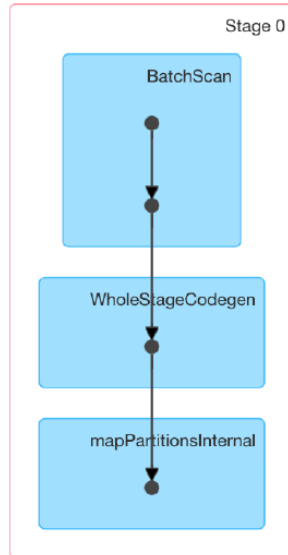
Status: SUCCEEDED

Associated SQL Query: 0

Completed Stages: 1

▸ Event Timeline

▼ DAG Visualization



▼ Completed Stages (1)

So what are RDDs?

The RDD is the most basic abstraction in Spark. There are three vital characteristics associated with an RDD:

- Dependencies
- Partitions (with some locality information)
- Compute function: $\text{Partition} \Rightarrow \text{Iterator}[T]$

So what are RDDs?

- First, a list of dependencies that instructs Spark how an RDD is constructed with its inputs is required. When necessary to reproduce results, Spark can recreate an RDD from these dependencies and replicate operations on it. This characteristic gives RDDs **resiliency**.
- Second, partitions provide Spark the ability to split the work to parallelize computation on partitions across executors. In some cases—for example, reading from HDFS—Spark will use locality information to send work to executors close to the data. That way less data is transmitted over the network.
- finally, an RDD has a compute function that produces an `Iterator[T]` for the data that will be stored in the RDD.

Working with Spark

- An RDD is a specification or recipe for creating a dataset. By knowing the inputs, their locations, and a compute function that produces an iterator, Spark can always recreate the dataset. This makes RDDs fault-tolerant and efficient.
- With the iterator being generic, Spark cannot do optimizations, since it is a “lambda” function → Solution is to Structure Spark thru APIs (**first with RDDs and then DSLs**)
- Therefore, Spark 2 onwards provided Domain Specific Language to do manipulations of the datasets, to enable it to do better optimization in addition to expressivity, simplicity, composability, and uniformity.

Working with Spark

```
val data = sc.parallelize(Seq("cat dog", "apple banana",  
"fish goat"), numSlices = 2)
```

```
val words = data.flatMap(line => line.split(" "))
```

Partition 0: "cat dog"
Partition 1: "apple banana", "fish goat"

flatMap

Partition 0: "cat", "dog"
Partition 1: "apple", "banana", "fish", "goat"

Working with Spark using RDD APIs

- # In Python

Create an RDD of tuples (name, age)

```
dataRDD = sc.parallelize([("Brooke", 20), ("Denny", 31), ("Jules", 30),  
("TD", 35), ("Brooke", 25)])
```

Use map and reduceByKey transformations with their lambda

expressions to aggregate and then compute average

```
agesRDD = (dataRDD.map(lambda x: (x[0], (x[1], 1)))  
.reduceByKey(lambda x, y: (x[0] + y[0], x[1] + y[1]))  
.map(lambda x: (x[0], x[1][0]/x[1][1])))
```

Working with Spark using RDD APIs

Map Function:

```
.map(lambda x: (x[0], (x[1], 1)))
```

Input (x): Each tuple in the form (name, age).

Transformation:

Converts (name, age) into (name, (age, 1)).

The (age, 1) pair tracks the sum of ages and the number of occurrences for each name.

Output:

```
[("Brooke", (20, 1)), ("Denny", (31, 1)), ("Jules", (30, 1)), ("TD", (35, 1)), ("Brooke", (25, 1))]
```

Working with Spark using RDD APIs

reduceByKey Function (`.reduceByKey(lambda x, y: (x[0] + y[0], x[1] + y[1]))`)

Purpose: Aggregate the data by key (name) to compute the total age and total count for each name.

How reduceByKey Works:

- Groups all values with the same key (name).
- Applies the provided function to merge values.

Input (x, y): x and y are the (age_sum, count) pairs for the same key. Example: For "Brooke", x = (20, 1) and y = (25, 1).

Transformation: Adds the ages and counts: `x[0] + y[0]`: Adds the age values (e.g., $20 + 25 = 45$). `x[1] + y[1]`: Adds the counts (e.g., $1 + 1 = 2$).

Output : `[("Brooke", (45, 2)), ("Denny", (31, 1)), ("Jules", (30, 1)), ("TD", (35, 1))]`

Working with Spark using RDD APIs

Second Map: `.map(lambda x: (x[0], x[1][0]/x[1][1]))`

Purpose: Compute the average age for each name.

Input (x): Each tuple in the form (name, (age_sum, count)).

Transformation:

Computes the average: `age_sum / count`.

Converts (name, (age_sum, count)) into (name, average_age).

Output: `[("Brooke", 22.5), ("Denny", 31.0), ("Jules", 30.0), ("TD", 35.0)]`

Working with Spark using RDD APIs

reduceByKey Function (`.reduceByKey(lambda x, y: (x[0] + y[0], x[1] + y[1]))`)

Purpose: Aggregate the data by key (name) to compute the total age and total count for each name.

How reduceByKey Works:

- Groups all values with the same key (name).
- Applies the provided function to merge values.

Input (x, y): x and y are the (age_sum, count) pairs for the same key. Example: For "Brooke", x = (20, 1) and y = (25, 1).

Transformation: Adds the ages and counts: x[0] + y[0]: Adds the age values (e.g., 20 + 25 = 45). x[1] + y[1]: Adds the counts (e.g., 1 + 1 = 2).

Output : [("Brooke", (45, 2)), ("Denny", (31, 1)), ("Jules", (30, 1)), ("TD", (35, 1))]

Working with Spark using RDD APIs

Summary

Function	Purpose	Transformation
First map	Attach a count to each age for aggregation.	$(\text{name}, \text{age}) \rightarrow (\text{name}, (\text{age}, 1))$
reduceByKey	Aggregate ages and counts for each name.	$(\text{name}, (\text{age_sum}, \text{count}))$ by summing values.
Second map	Compute the average age for each name.	$(\text{name}, (\text{age_sum}, \text{count})) \rightarrow (\text{name}, \text{average_age})$

This code, which tells Spark how to aggregate keys and compute averages with a string of lambda functions, is cryptic and hard to read. In other words, the code is instructing Spark how to compute the query. It's completely opaque to Spark, because it doesn't communicate the intention.

Src: Learning Spark

© Walid Hassan, M.B.A, D.Eng.

Working with Spark using DSL APIs

In Python

```
from pyspark.sql import SparkSession
```

```
from pyspark.sql.functions import avg
```

Create a DataFrame using SparkSession

```
spark = (SparkSession
```

```
.builder
```

```
.appName("AuthorsAges")
```

```
.getOrCreate())
```

Create a DataFrame

```
data_df = spark.createDataFrame([("Brooke", 20), ("Denny", 31), ("Jules", 30), ("TD", 35), ("Brooke", 25)], ["name", "age"])
```

Group the same names together, aggregate their ages, and compute an average

```
avg_df = data_df.groupBy("name").agg(avg("age"))
```

Show the results of the final execution

```
avg_df.show()
```

Src: Learning Spark

Output

name	avg(age)
Brooke	22.5
Jules	30.0
TD	35.0
Denny	31.0

Working with Spark using DSL APIs

- This version of the code is far more expressive as well as simpler than the earlier version, because we are using high-level DSL operators and APIs to tell Spark what to do.
- In effect, we have employed these operators to compose our query. Spark can inspect or parse this query and understand our intention, it can optimize or arrange the operations for efficient execution. Spark knows exactly what we wish to do: group people by their names, aggregate their ages, and then compute the average age of all people with the same name.
- All of this simplicity and expressivity that we developers cherish is possible because of the Spark SQL engine upon which the high-level Structured APIs are built. It is because of this engine, which underpins all the Spark components, that we get uniform APIs.

Src: Learning Spark

© Walid Hassan, M.B.A, D.Eng.

Working with Spark using DSL APIs

- While RDDs can still be used (actually, they are the underlying APIs behind the high level structured APIs), there is rarely a need to do so.
- Spark supports multiple languages: Python, Scala, R, and Java.
- We will be focused on Python & DataFrame APIs
- DataFrames are immutable and Spark keeps a lineage of all transformations. You can add or change the names and data types of the columns, creating new DataFrames while the previous versions are preserved

Data Types - Basic

- Matching its supported programming languages, Spark supports basic internal data types.

These data types can be declared in your Spark application or defined in your schema.

Data type	Value assigned in Python	API to instantiate
ByteType	int	DataTypes.ByteType
ShortType	int	DataTypes.ShortType
IntegerType	int	DataTypes.IntegerType
LongType	int	DataTypes.LongType
FloatType	float	DataTypes.FloatType
DoubleType	float	DataTypes.DoubleType
StringType	str	DataTypes.StringType
BooleanType	bool	DataTypes.BooleanType
DecimalType	decimal.Decimal	DecimalType

Data Types - Complex

- maps, arrays, structs, dates, timestamps, fields, etc..

Data type	Value assigned in Python	API to instantiate
BinaryType	bytearray	BinaryType()
TimestampType	datetime.datetime	TimestampType()
DateType	datetime.date	DateType()
ArrayType	List, tuple, or array	ArrayType(dataType, [nullable])
MapType	dict	MapType(keyType, valueType, [nullable])
StructType	List or tuple	StructType([fields])
StructField	A value type corresponding to the type of this field	StructField(name, dataType, [nullable])

Src: Learning Spark

© Walid Hassan, M.B.A, D.Eng.

Data Types - Complex

In Apache Spark, StructType and StructField are used to define the schema of a DataFrame or Dataset when working with structured data. They are part of Spark's pyspark.sql.types module and allow you to specify the structure (schema) of your data programmatically.

What is StructField? StructField defines a single column in the schema. It specifies: The column name.

The data type (e.g., IntegerType, StringType).

Whether the column allows null values (nullable).

What is StructType? StructType is a collection of StructField objects.

It represents the schema of a DataFrame, defining the columns, their data types, and whether they can contain null values.

Schemas

A schema in Spark defines the column names and associated data types for a Data- Frame.

Most often, schemas come into play when you are reading structured data from an external data source (more on this in the next chapter). Defining a schema up front as opposed to taking a schema-on-read approach offers three benefits:

- You relieve Spark from the onus of inferring data types.
- You prevent Spark from creating a separate job just to read a large portion of your file to ascertain the schema, which for a large data file can be expensive and time-consuming.
- You can detect errors early if data doesn't match the schema.

→ always define your schema up front whenever you want to read a large file from a data source

Schemas

Spark allows you to define a schema in two ways. One is to define it programmatically, and the other is to employ a Data Definition Language (DDL) string, which is much simpler and easier to read.

To define a schema programmatically for a DataFrame with three named columns, author, title, and pages, you can use the Spark DataFrame API.

Programmatically:

```
# In Python
```

```
from pyspark.sql.types import *
```

```
schema = StructType([StructField("author", StringType(), False),
```

```
StructField("title", StringType(), False),
```

```
StructField("pages", IntegerType(), False)])
```

Src: Learning Spark

© Walid Hassan, M.B.A, D.Eng.

Schemas

Data Definition Language (DDL)

Defining the same schema using DDL is much simpler:

In Python

```
schema = "author STRING, title STRING, pages INT"
```

You can choose whichever way you like to define a schema.

Rows:

A row in Spark is a generic Row object, containing one or more columns. Each column may be of the same data type (e.g., integer or string), or they can have different types (integer, string, map, array, etc.). Because Row is an object in Spark and an ordered collection of fields, you can instantiate a Row in each of Spark's supported languages and access its fields by an index starting at 0..

In Python

```
from pyspark.sql import Row
blog_row = Row(6, "Reynold", "Xin",
               "https://tinyurl.6", 255568, "3/2/2015",
               ["twitter", "LinkedIn"])
# access using index for individual items
blog_row[1]
'Reynold'
```

In Python

```
rows = [Row("Matei Zaharia", "CA"),
        Row("Reynold Xin", "CA")]
authors_df = spark.createDataFrame(rows,
                                   ["Authors", "State"])
authors_df.show()
```

Author	State
Matei Zaharia	CA
Reynold Xin	CA

Common DataFrame Operations - Reading

- To perform common data operations on DataFrames, you'll first need to load a DataFrame from a data source that holds your structured data.
- DataFrameReader, that enables you to read data into a DataFrame from myriad data sources in formats such as JSON, CSV, Parquet, Text, Avro, ORC, etc.
- For Large data, please provide the schema to relieve Spark from inferring it.

Use the DataFrameReader interface to read a CSV file

```
sf_fire_file = "/databricks-datasets/learning-spark-v2/sf-fire/sf-fire-calls.csv"  
fire_df = spark.read.csv(sf_fire_file, header=True, schema=fire_schema)
```

Common DataFrame Operations - Writing

- DataFrameWriter interface (supports multiple data sources).
- Parquet, a popular columnar format, is the default format; it uses snappy compression to compress the data.
- If the DataFrame is written as Parquet, the schema is preserved as part of the Parquet metadata. In this case, subsequent reads back into a DataFrame do not require you to manually supply a schema For Large data, please provide the schema to relieve Spark from inferring it.

In Python to save as a Parquet file

```
parquet_path = ...  
fire_df.write.format("parquet").save(parquet_path)
```

In Python

```
parquet_table = ... # name of the table  
fire_df.write.format("parquet").saveAsTable(parquet_table)
```

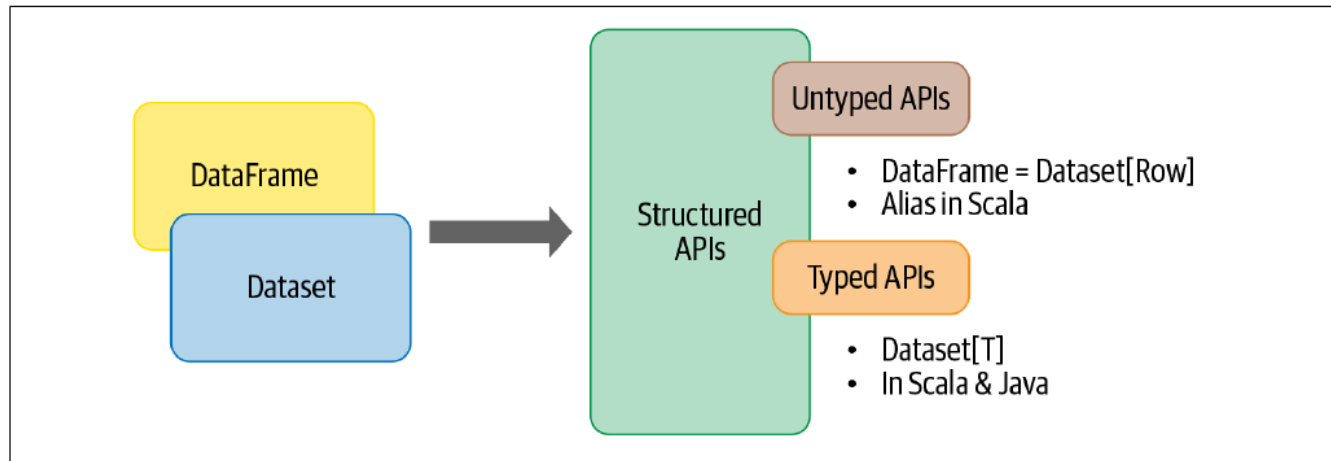
Managed vs. Un-Managed Tables

- Spark allows you to create two types of tables: **managed and unmanaged**. For a managed table, Spark manages both the metadata and the data in the file store. This could be a local filesystem, HDFS, or an object store such as Amazon S3 or Azure Blob.
- For an unmanaged table, Spark only manages the metadata, while you manage the data yourself in an external data source such as Cassandra. With a managed table, because Spark manages everything, a SQL command such as `DROP TABLE table_name` deletes both the metadata and the data. With an unmanaged table, the same command will delete only the metadata, not the actual data.

Common DataFrame Operations

- **Transformations and actions**
- **Projections and filters.** A projection in relational parlance is a way to return only the rows matching a certain relational condition by using filters. In Spark, projections are done with the `select()` method, while filters can be expressed using the `filter()` or `where()` method.
- **Renaming, adding, and dropping columns.** Sometimes you want to rename particular columns for reasons of style or convention, and at other times for readability or brevity.
- `spark.sql.functions` has a set of to/from date/timestamp functions such as `to_timestamp()` and `to_date()` that we can use for manipulating date formats
- **Aggregations.** A handful of transformations and actions on DataFrames, such as `groupBy()`, `orderBy()`, and `count()`, offer the ability to aggregate by column names and then aggregate counts across them

Dataset API



- Spark 2.0 unified the DataFrame and Dataset APIs as Structured APIs with similar interfaces so that developers would only have to learn a single set of APIs. Datasets take on two characteristics: typed and untyped APIs. A Dataset, is a collection of strongly typed JVM objects in Scala or a class in Java
- In Spark's supported languages, Datasets make sense only in Java and Scala, whereas in Python and R only DataFrames make sense. This is because Python and R are not compile-time type-safe; types are dynamically inferred or assigned during execution, not during compile time. The reverse is true in Scala and Java

Src: Learning Spark

© Walid Hassan, M.B.A, D.Eng.

DataFrame vs. Datasets

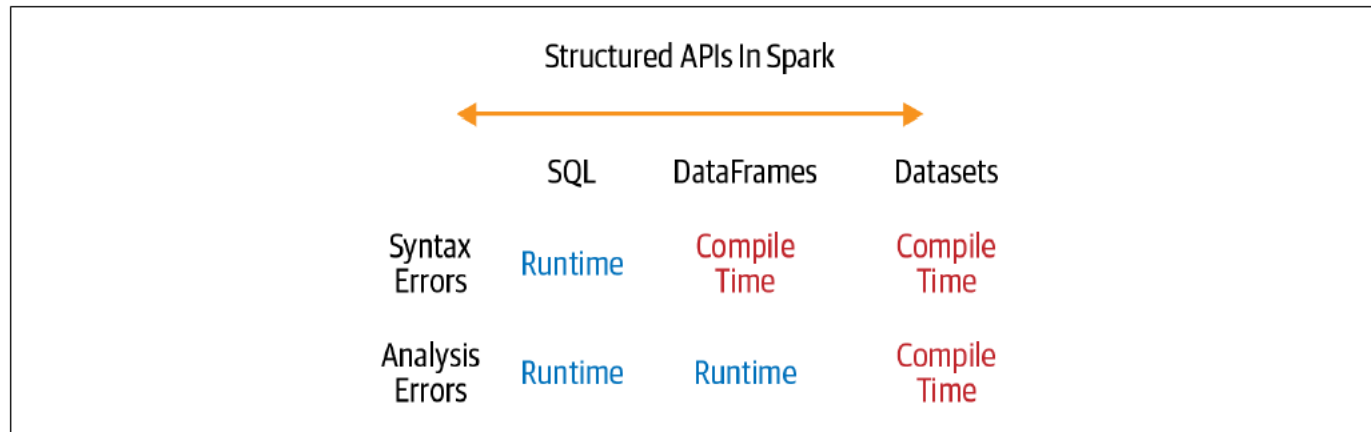
- If you want to tell Spark what to do, not how to do it, use DataFrames or Datasets.
- If you want rich semantics, high-level abstractions, and DSL operators, use Data-Frames or Datasets.
- If you want strict compile-time type safety and don't mind creating multiple case classes for a specific Dataset[T], use Datasets.
- If your processing demands high-level expressions, filters, maps, aggregations, computing averages or sums, SQL queries, columnar access, or use of relational operators on semi-structured data, use DataFrames or Datasets.
- If your processing dictates relational transformations similar to SQL-like queries, use DataFrames.
- If you want to take advantage of and benefit from Tungsten's efficient serialization with Encoders, use Datasets.
- If you want unification, code optimization, and simplification of APIs across Spark components, use DataFrames.

Src: Learning Spark

© Walid Hassan, M.B.A, D.Eng.

DataFrame vs. Datasets

- If you are an R user, use DataFrames.
- If you are a Python user, use DataFrames and drop down to RDDs if you need more control.
- If you want space and speed efficiency, use DataFrames.
- If you want errors caught during runtime vs. compilation, choose the appropriate API



Src: Learning Spark

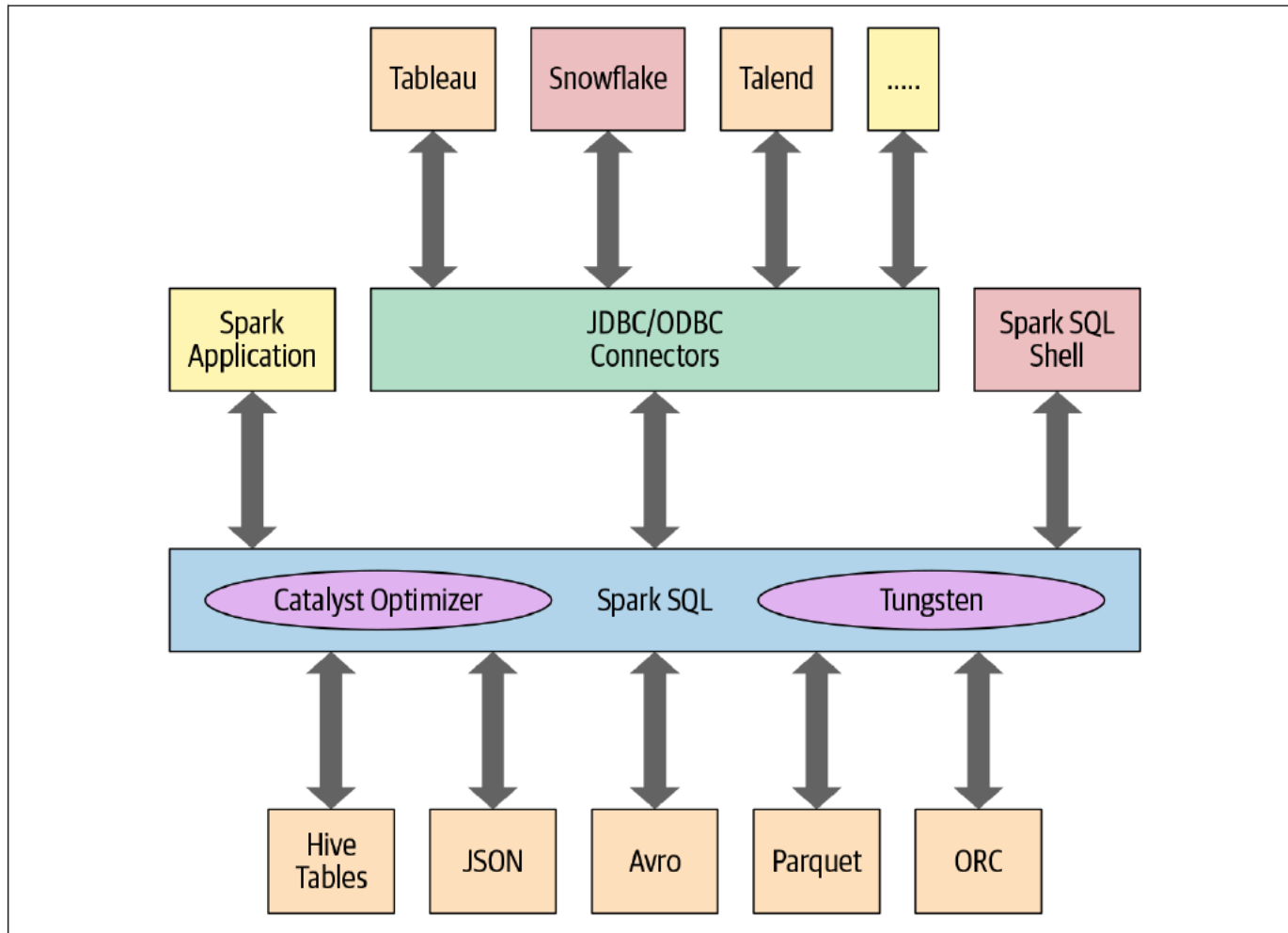
© Walid Hassan, M.B.A, D.Eng.

RDD APIs

There are some scenarios where you'll want to consider using RDDs, such as when you:

- Are using a third-party package that's written using RDDs
- Can forgo the code optimization, efficient space utilization, and performance benefits available with DataFrames and Datasets
- Want to precisely instruct Spark how to do a query
- you can seamlessly move between DataFrames or Datasets and RDDs at will using a simple API method call, `df.rdd`. (Note, however, that this does have a cost and should be avoided unless necessary.)

Spark SQL & Underlying Engine



Src: Learning Spark

© Walid Hassan, M.B.A, D.Eng.

SQL Engine

At the core of the **Spark SQL engine** are the **Catalyst optimizer** and **Project Tungsten**.

Together, these support the high-level DataFrame and Dataset APIs and SQL queries.

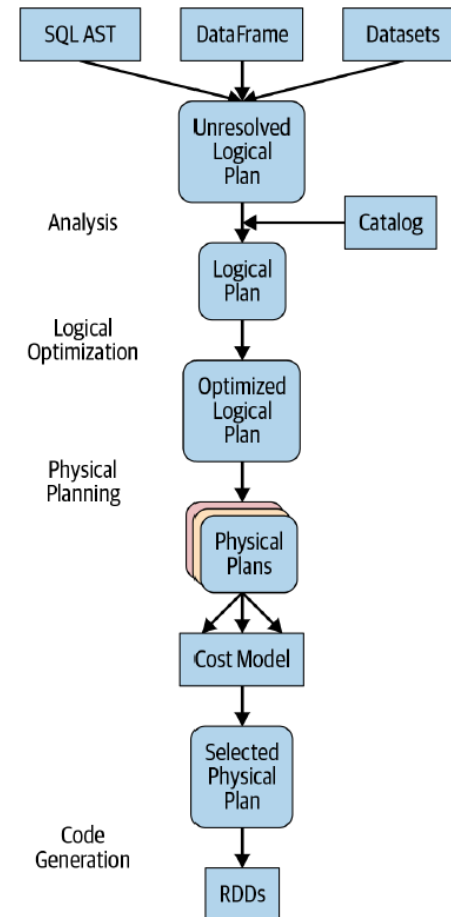
The Spark SQL engine:

- Unifies Spark components and permits abstraction to DataFrames/Datasets in Java, Scala, Python, and R, which simplifies working with structured data sets.
- Connects to the Apache Hive metastore and tables.
- Reads and writes structured data with a specific schema from structured file formats (JSON, CSV, Text, Avro, Parquet, ORC, etc.) and converts data into temporary tables.
- Offers an interactive Spark SQL shell for quick data exploration.
- Provides a bridge to (and from) external tools via standard database JDBC/ ODBC connectors.
- Generates optimized query plans and compact code for the JVM, for final execution.

Catalyst Optimizer

The Catalyst optimizer takes a computational query and converts it into an execution plan. It goes through four transformational phases

1. Analysis
2. Logical optimization
3. Physical planning
4. Code generation



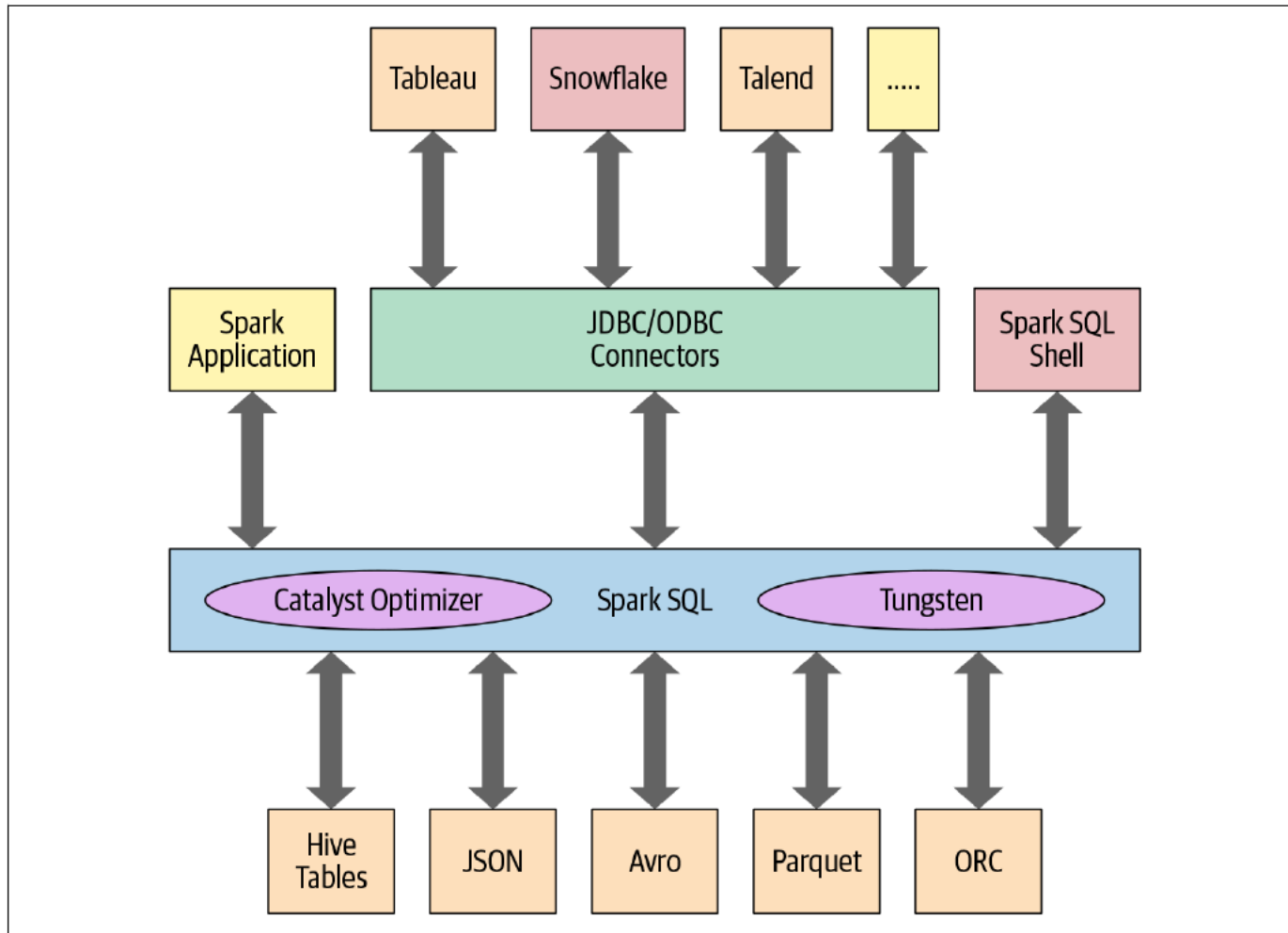
Src: Learning Spark

© Walid Hassan, M.B.A, D.Eng.

Catalyst Optimizer

1. Analysis: In this initial phase, any columns or table names will be resolved by consulting an internal Catalog, a programmatic interface to Spark SQL that holds a list of names of columns, data types, functions, tables, databases
2. Logical optimization: Catalyst optimizer will first construct a set of multiple plans and then, using its cost-based optimizer (CBO), assign costs to each plan. These plans are laid out as operator trees.
3. Physical planning: Spark SQL generates an optimal physical plan for the selected logical plan, using physical operators that match those available in the Spark execution engine.
4. Code generation: The final phase of query optimization involves generating efficient Java bytecode to run on each machine.

Spark SQL & Underlying Engine



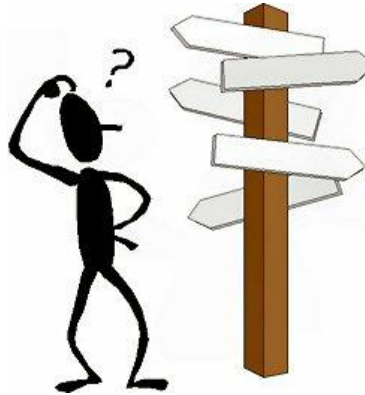
Src: Learning Spark

© Walid Hassan, M.B.A, D.Eng.

Dataset API

- Similar to DataFrames, you can perform similar actions/operations on datasets.

DataFrame



Dataset

Spark APIs

- <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/functions.html>

Commonly Used Functions

Transformations

- `select`: Select specific columns.
- `filter`: Filter rows based on a condition.
 - example: `mn_df.filter(mn_df["State"] == "CA")`
- `groupBy`: Group data for aggregations.
- `agg`: Perform aggregate functions on grouped data.
- `withColumn`: Add or transform columns.
 - example: `mn_df.withColumn("DoubleCount", mn_df["Count"] * 2)`
- `orderBy`: Sort rows by one or more columns.
- `distinct`: Remove duplicate rows.
- `join`: Combine two DataFrames.
 - example: `df1.join(df2, on="key_column", how="inner")`

Commonly Used Functions

Actions

- `show`: Display rows of the DataFrame.
- `count`: Get the number of rows.
- `collect`: Retrieve all data to the driver (use with caution for large datasets).
- `take(n)`: Retrieve the first n rows.
- `write`: Save the DataFrame to a file or table.



Hands On.....

Databricks file commands

DBUtils: <https://docs.databricks.com/en/dev-tools/databricks-utils.html>



BREAK



**Please come back @
2:34 PM EST
1:34 PM CST**

References

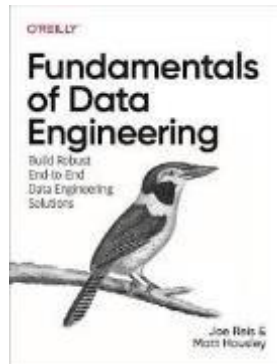
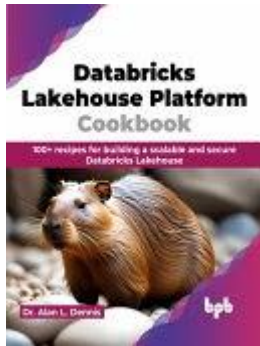
In addition to the references in each slide the below are leveraged throughout this course.

Gareth James, D. W. (2023). An Introduction to Statistical Learning: with Applications in Python. Springer.

(Jules S. Damji, July 2020), Learning Spark Lightning Fast Data Analytics , 2020 (2nd Edition).

Databricks Lakehouse Platform Cookbook, Dennis, Dr. Alan L.

Fundamentals of Data Engineering, O'Reilly (J. Reis, M. Housley)

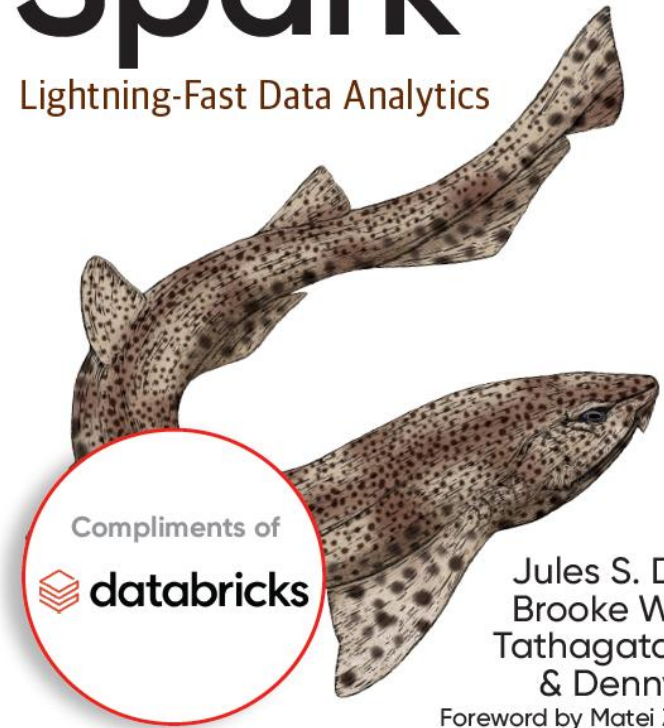


O'REILLY®

Learning Spark

Lightning-Fast Data Analytics

2nd Edition
Covers
Apache Spark 3.0



Jules S. Damji,
Brooke Wenig,
Tathagata Das
& Denny Lee
Foreword by Matei Zaharia

© Walid Hassan, M.B.A, D.Eng.

THE GEORGE
WASHINGTON
UNIVERSITY
WASHINGTON, DC